CPU Scheduling Part - I

Foundation Course on Operating Systems for GATE 2022

Sanchit Jain • Lesson 5 • July 26, 2021

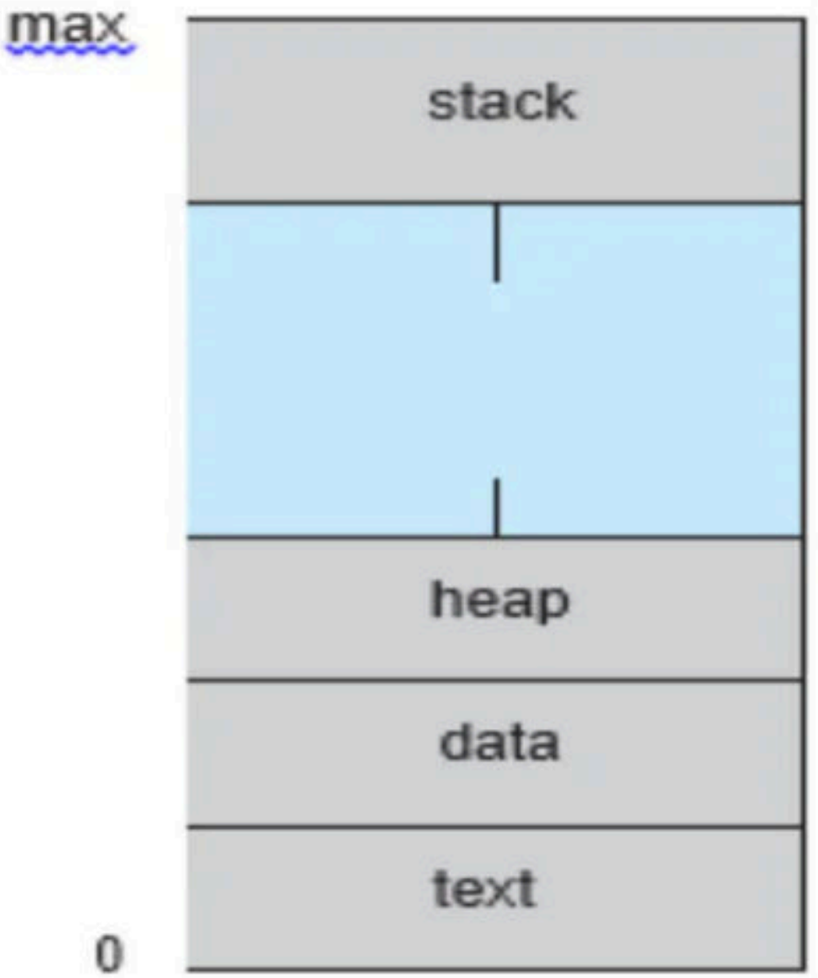
Process

- In general, a process is a program in execution.
- A Program is not a Process by default. A program is a passive entity, i.e. a file containing a list of instructions stored on disk (secondary memory) (often called an executable file).
- A program becomes a Process when an executable file is loaded into main memory and when it's PCB is created.
- A process on the other hand is an Active Entity, which require resources like main memory, CPU time, registers, system buses etc.

- Even if two processes may be associated with same program, they will be considered as two separate execution sequences and are totally different process.

- For instance, if a user has invoked many copies of web browser program, each copy will be treated as separate process. even though the text section is same but the data, heap and stack sections can vary.

- A Process consists of following sections:
  - **Text section:** also known as Program Code.
  - **Stack:** which contains the temporary data (Function Parameters, return addresses and local variables).
  - **Data Section:** containing global variables.
  - **Heap:** which is memory dynamically allocated during process runtime.



Q Process is a: **(ISRO 2009)**

a) A program in high level language kept on disk

b) Contents of main memory

c) A program in execution

d) A job in secondary memory

Break

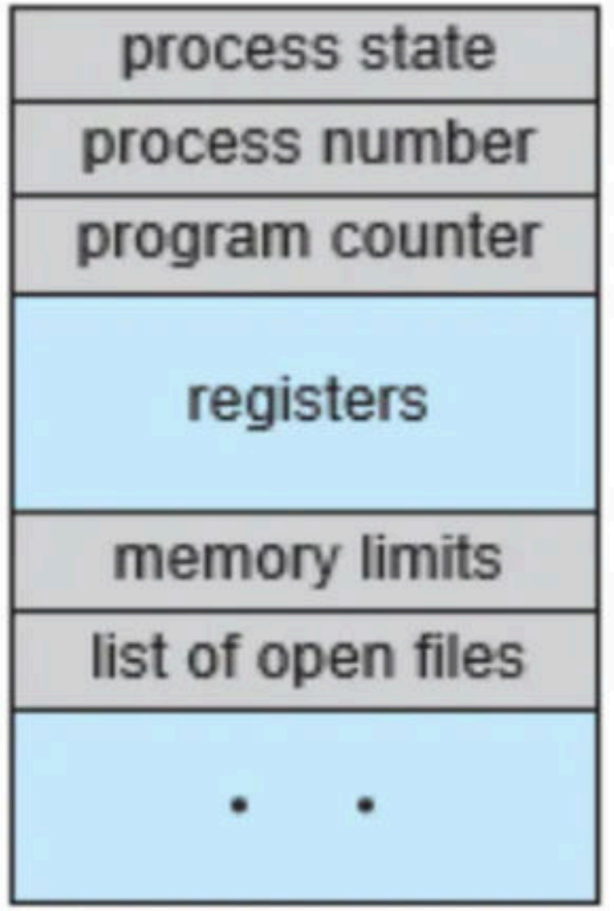
Process Control Block (PCB)

- Each process is represented in the operating system by a process control block (PCB) — also called a task control block.

- PCB simply serves as the repository for any information that may vary from process to process. It contains many pieces of information associated with a specific process, including these:
  - **Process state:** The state may be new, ready, running, waiting, halted, and so on.

  - **Program counter:** The counter indicates the address of the next instruction to be executed for this process.

  - **CPU registers:** The registers vary in number and type, depending on the computer architecture. They include accumulators, index registers, stack pointers, and general-purpose registers, plus any condition-code information. Along with the program counter, this state information must be saved when an interrupt occurs, to allow the process to be continued correctly afterward.

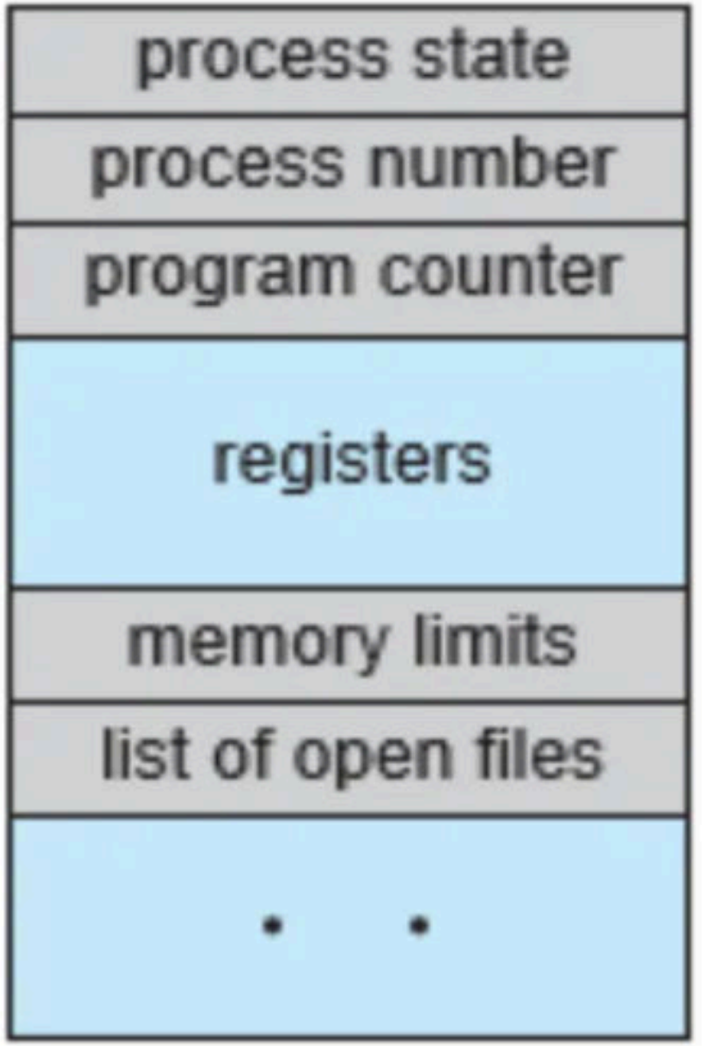


- **CPU-scheduling information:** This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.

- **Memory-management information:** This information may include such items as the value of the base and limit registers and the page tables, or the segment tables, depending on the memory system used by the operating system.

- **Accounting information:** This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.

- **I/O status information:** This information includes the list of I/O devices allocated to the process, a list of open files, and so on.

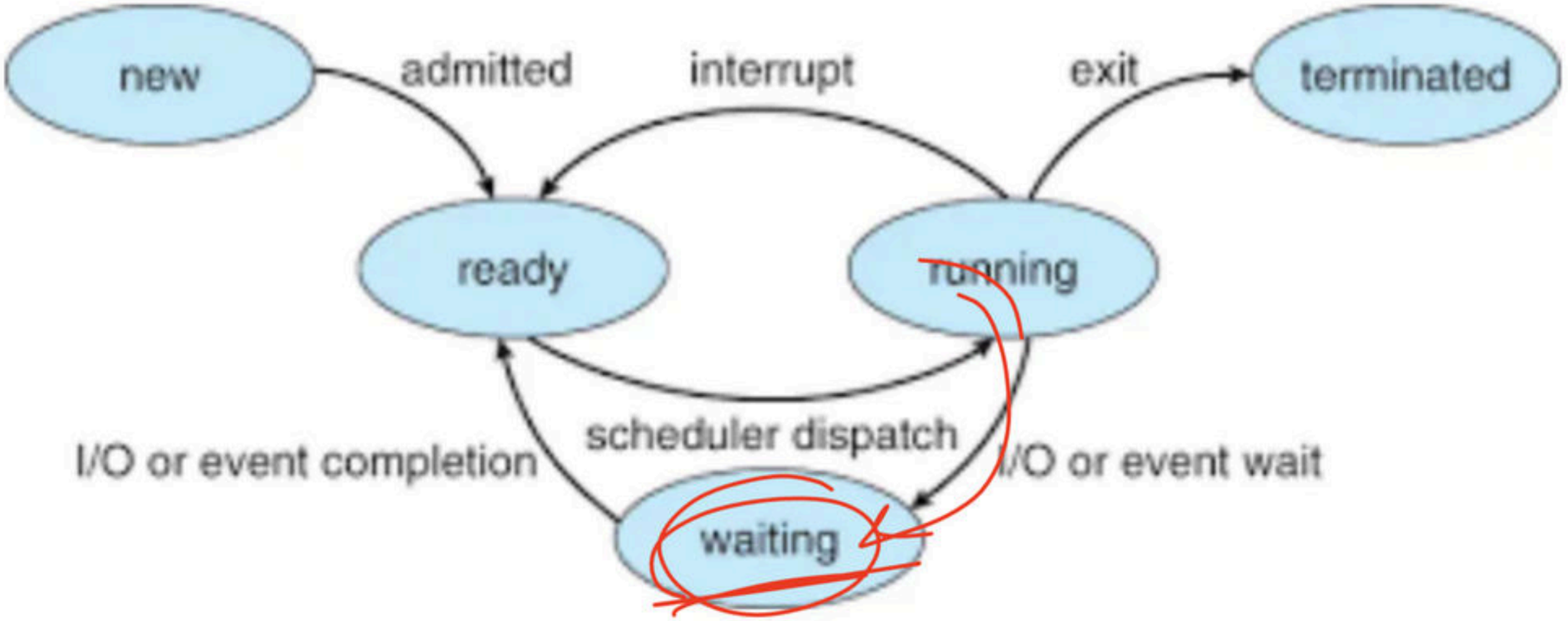


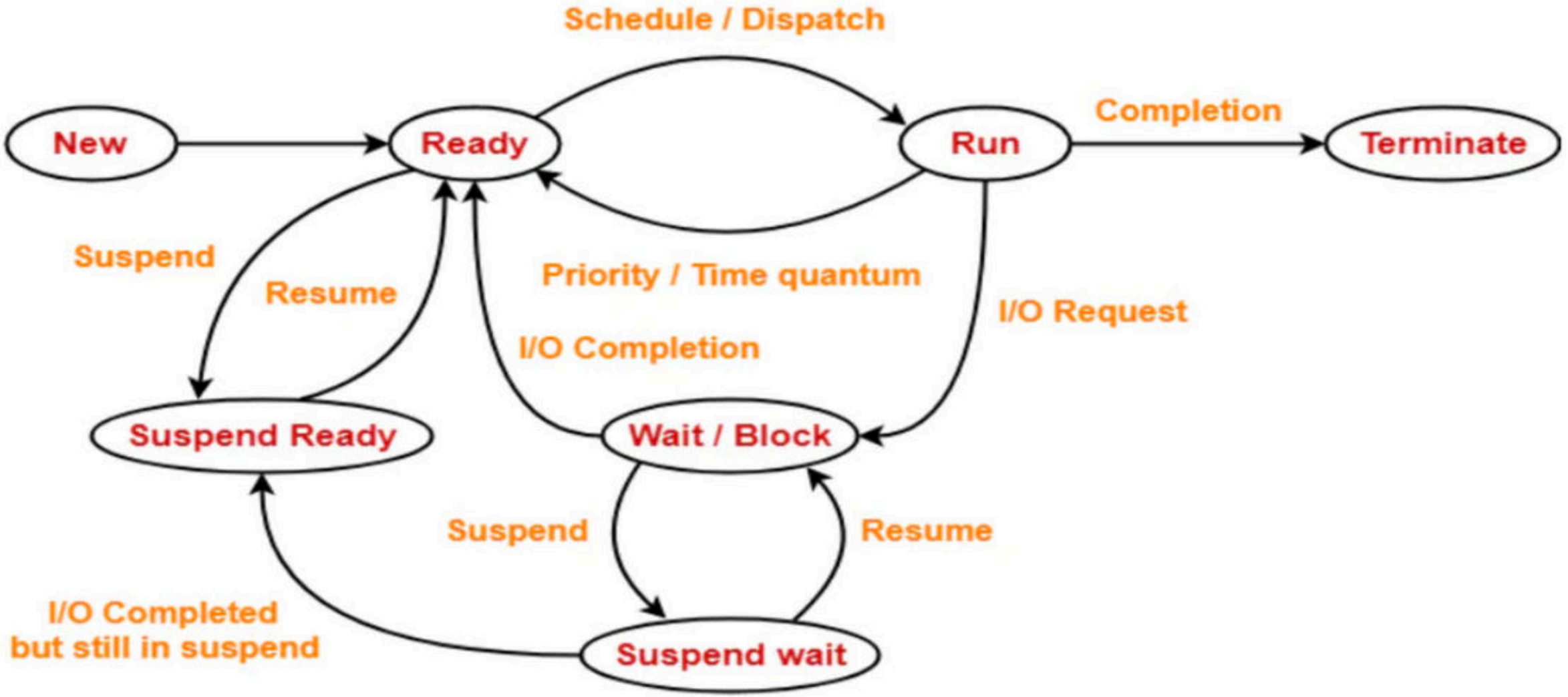
Break

The Human Life Cycle

Process States

- A Process changes states as it executes. The state of a process is defined in parts by the current activity of that process. A process may be in one of the following states:
  - **New:** The process is being created.
  - **Running:** Instructions are being executed.
  - **Waiting (Blocked):** The process is waiting for some event to occur (such as an I/O completion or reception of a signal).
  - **Ready:** The process is waiting to be assigned to a processor.
  - **Terminated:** The process has finished execution.


new
admitted
interrupt
exit
terminated
ready
running
I/O or event completion
scheduler dispatch
I/O or event wait
waiting



Process State Diagram

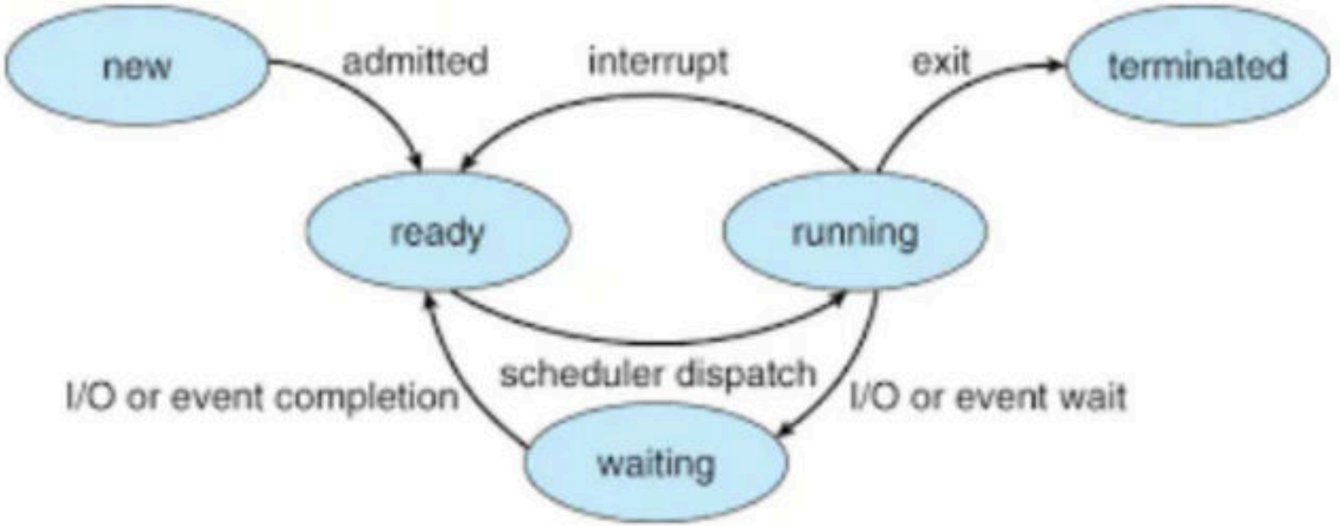
Break

Q How many states can a process be in? **(NET-DEC-2007)**

a) 2 **b)** 3 **c)** 4 **d)** 5

Q What is the minimum and maximum number of processes that can be in the ready, run, and blocked states, if total number of process is n?

	Min	Max
Ready		
Run		
Block		



Q On a system with P CPUs and n processes, what is the minimum and maximum number of processes that can be in the ready, run, and blocked states?

	Min	Max
Ready		
Run		
Block		

Q The state of a process after it encounters an I/O instruction is **(ISRO 2013)**

a) ready **b)** blocked **c)** idle **d)** running

Q There are three processes in the ready queue. When the currently running process requests for I/O how many process switches take place? **(ISRO 2011)**

a) 1 **b)** 2 **c)** 3 **d)** 4

Q What is the ready state of a process?

a) when process is scheduled to run after some execution

b) when process is using the CPU

c) when process is unable to run until some task has been completed

d) none of the mentioned

Q A task in a blocked state

a) is executable

b) is running

c) must still be placed in the run queues

d) is waiting for some temporarily unavailable resources

Break

Q Which combination of the following features will suffice to characterize an OS as a multi-programmed OS? **(GATE-2002) (2 Marks)**

(a) More than one program may be loaded into main memory at the same time for execution.

(b) If a program waits for certain events such as I/O, another program is immediately scheduled for execution.

(c) If the execution of program terminates, another program is immediately scheduled for execution.

(A) a **(B)** a and b **(C)** a and c **(D)** a, b and c

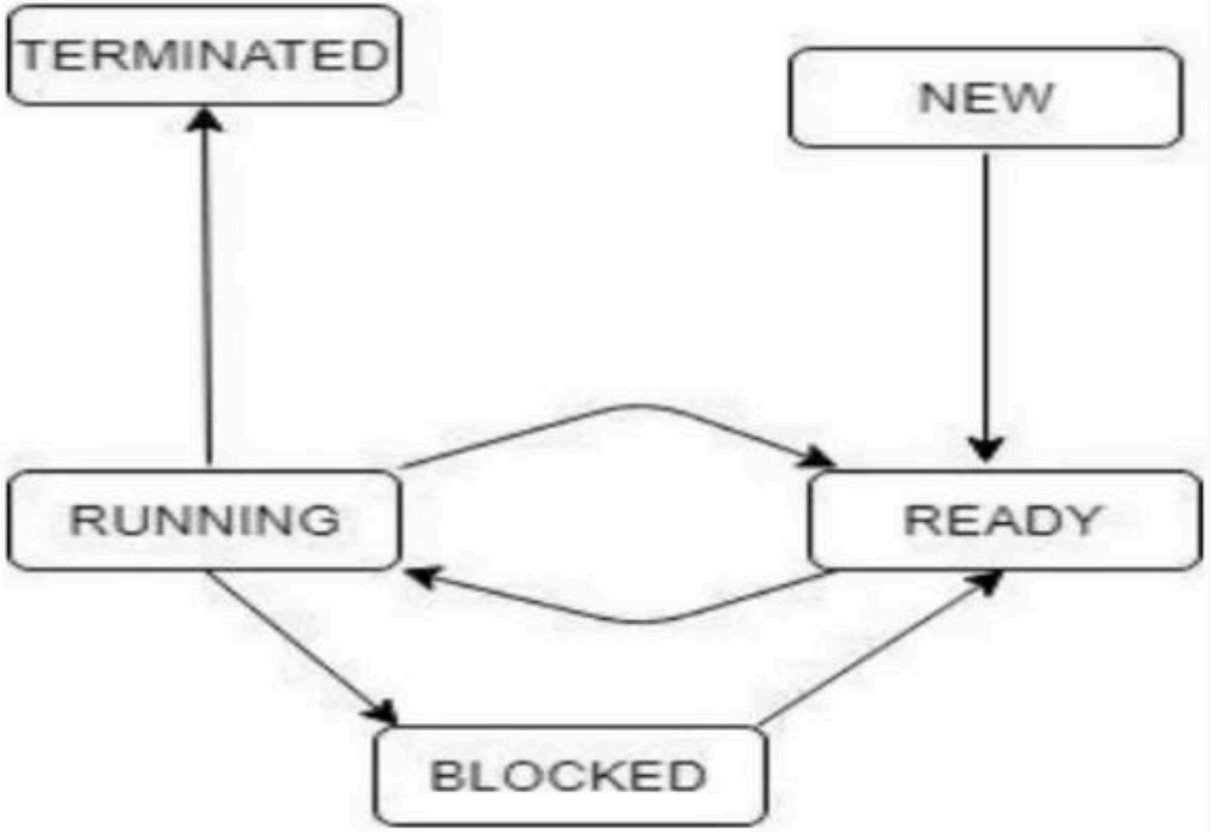
Q The process state transition diagram in below figure is representative of Untitled Diagram **(GATE-1996) (1 Marks)**

a) a batch operating system

b) an operating system with a preemptive schedular

c) an operating system with a non-preemptive schedular

d) a uni-programmed operating system



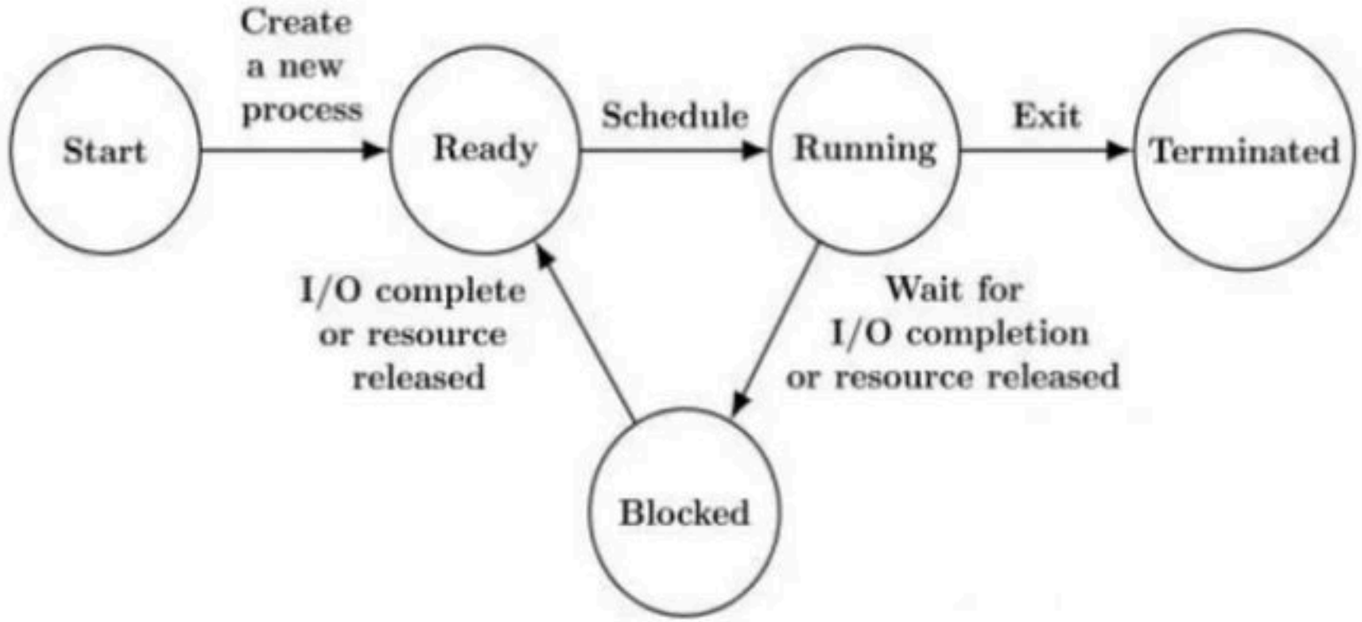
Q The process state transition diagram of an operating system is as given below. Which of the following must be FALSE about the above operating system? **(GATE-2006) (1 Marks)**

a) It is a multiprogram operating system

b) It uses preemptive scheduling

c) It uses non-preemptive scheduling

d) It is a multi-user operating system



Q In the following process state transition diagram for a uniprocessor system, assume that there are always some processes in the ready state:

Now consider the following statements:

I) If a process makes a transition D, it would result in another process making transition A immediately.

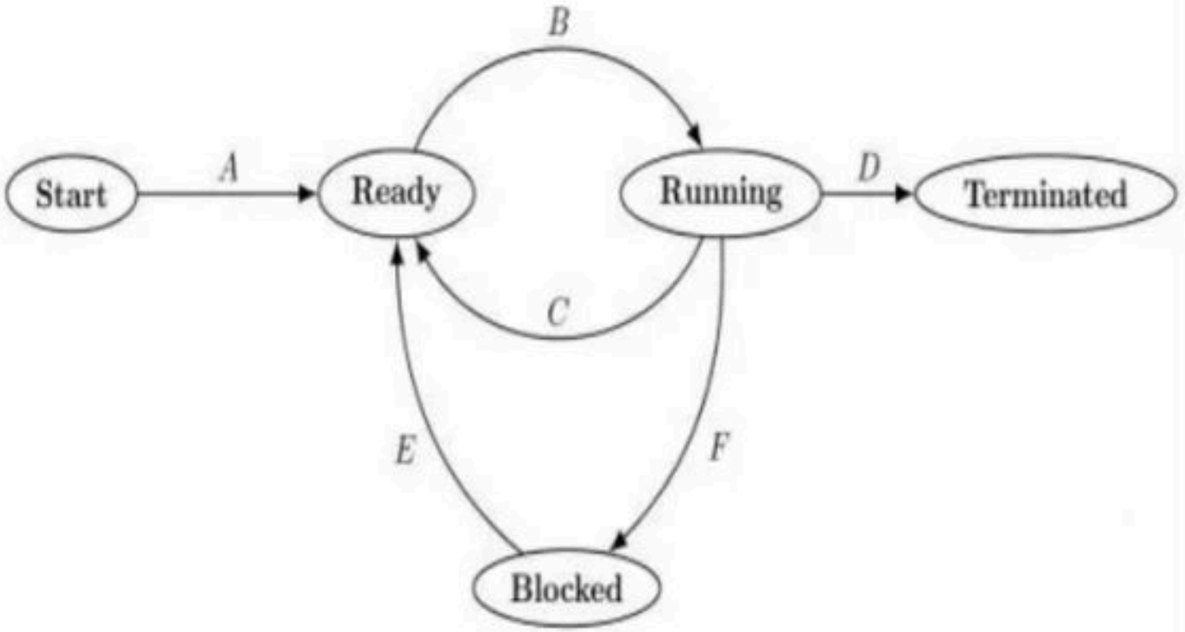
II) A process P2 in blocked state can make transition E while another process P1 is in running state.

III) The OS uses preemptive scheduling.

IV) The OS uses non-preemptive scheduling.

Which of the above statements are TRUE? **(GATE-2009) (2 Marks)**

a) I and II **b)** I and III **c)** II and III **d)** II and IV



Q A computer handles several interrupt sources of which the following are relevant for this question.

- Interrupt from CPU temperature sensor (raises interrupt if CPU temperature is too high)
- Interrupt from Mouse (raises interrupt if the mouse is moved or a button is pressed)
- Interrupt from Keyboard (raises interrupt when a key is pressed or released)
- Interrupt from Hard Disk (raises interrupt when a disk read is completed)

Which one of these will be handled at the HIGHEST priority? **(GATE - 2011) (1 Marks)**

(A) Interrupt from Hard Disk

(B) Interrupt from Mouse

(C) Interrupt from Keyboard

(D) Interrupt from CPU temperature sensor

Q Match the following: (NET-SEP-2013)

List – I	List - II
Process state	Reason for transition
a) Ready → Running	i. Request made by the process is satisfied or an event for which it was waiting occurs.
b) Blocked → Ready	ii. Process wishes to wait for some action by another process.
c) Running → Blocked	iii. The process is dispatched.
d) Running → Ready	iv. The process is pre-empted.

	a	b	c	d
a)	iii	i	ii	iv
b)	iv	i	iii	ii
c)	iv	iii	i	ii
d)	liii	iii	ii	i

73
11
8
8

Q Which is the correct definition of a valid process transition in an operating system? **(NET-JUNE-2005)**

a) Wake up : Ready → Running

b) Dispatch : Ready→ Running

c) Block: Ready → Running

d) Timer run out: Ready → Blocked

Break

- The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization.

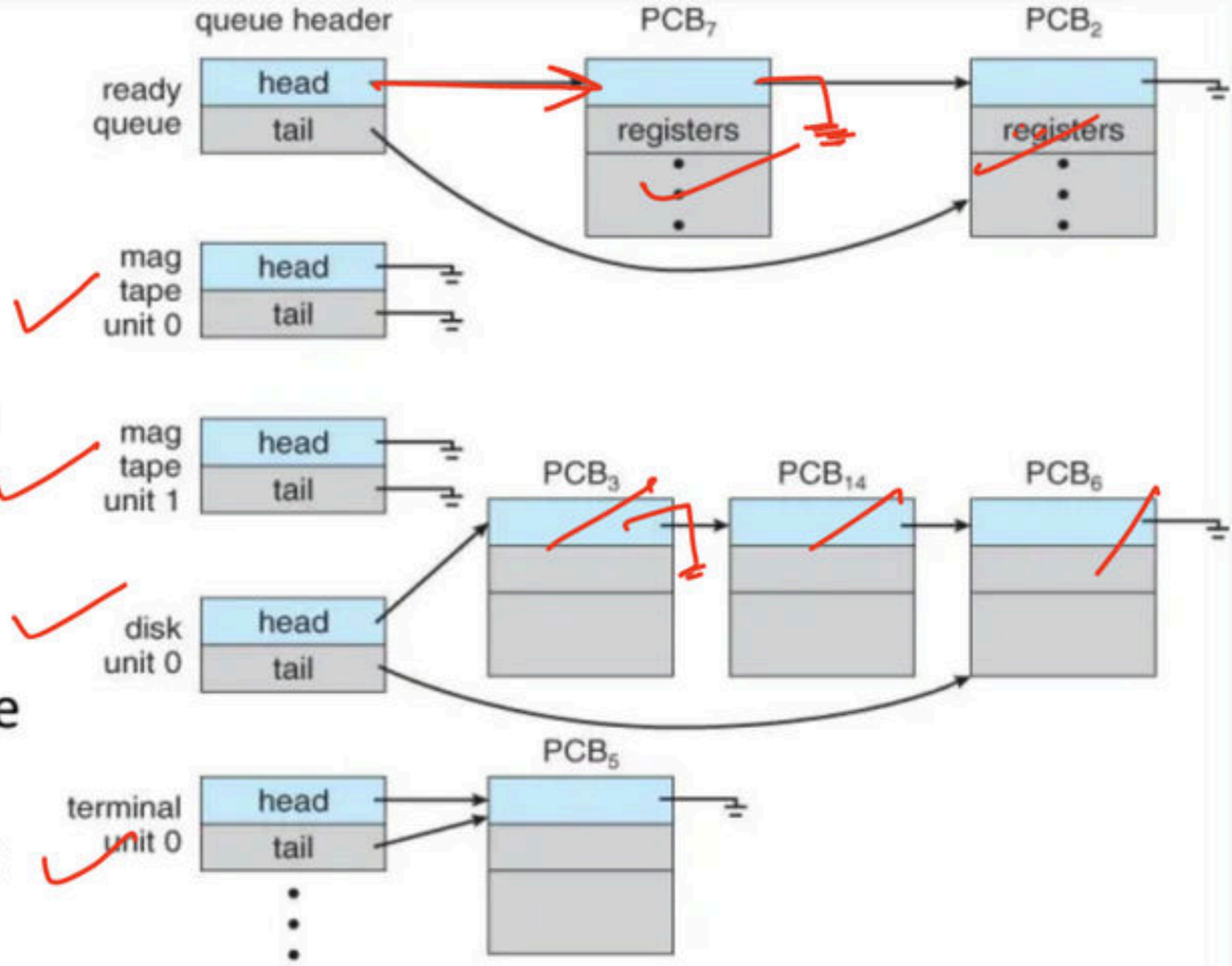
- The objective of time sharing is to switch the CPU among processes so frequently that users can interact with each program while it is running.

- To meet these objectives, the process scheduler selects an available process (possibly from a set of several available processes) for execution on the CPU. Note: For a single-processor system, there will never be more than one running process.

- Scheduling Queues - As processes enter the system, they are put into a job queue, which consists of all processes in the system.

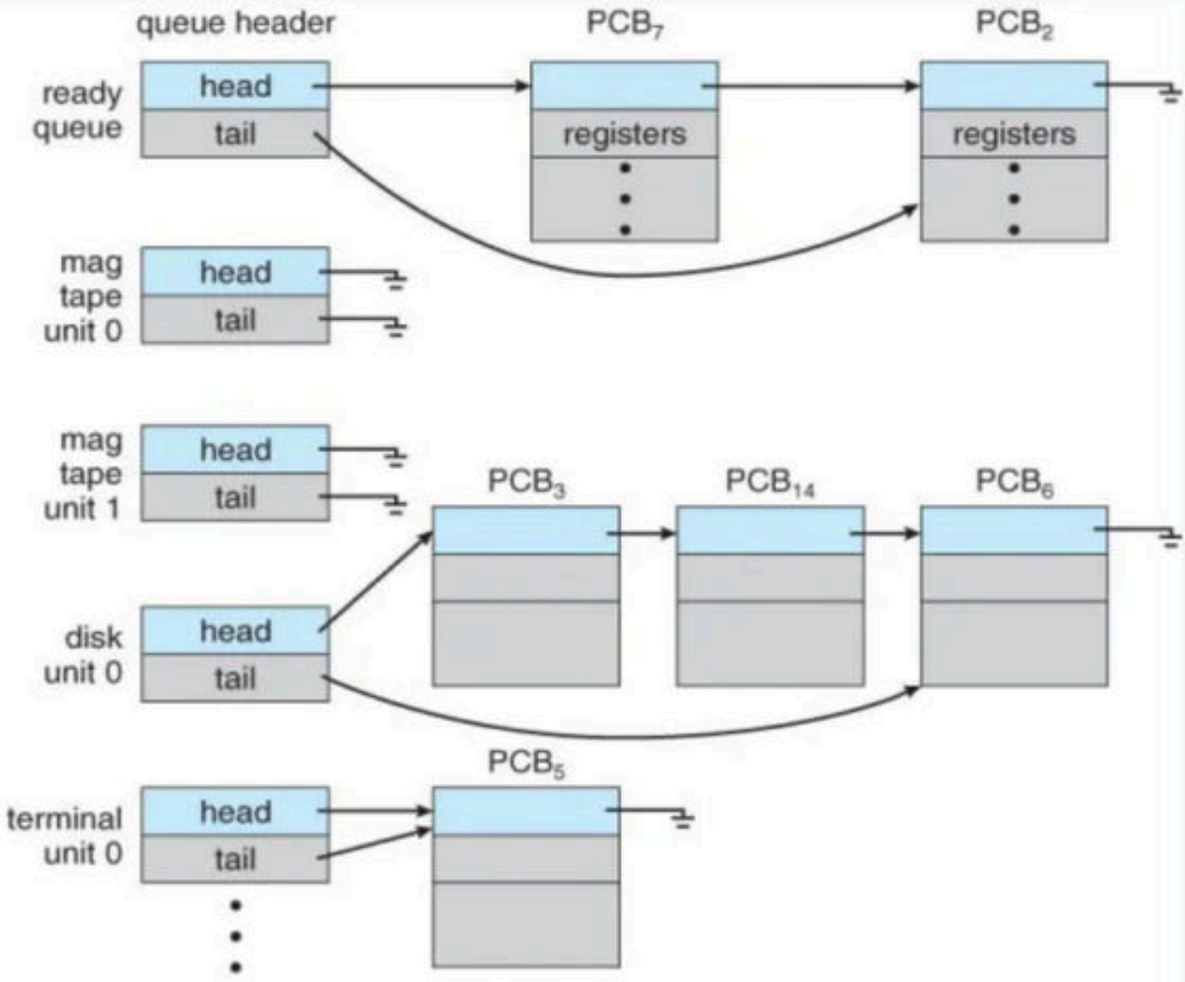
- The processes that are residing in main memory and are ready to execute are kept on a list called the ready queue. This queue is generally stored as a linked list.

- A ready-queue header contains pointers to the first and final PCBs in the list. Each PCB includes a pointer field that points to the next PCB in the ready queue.

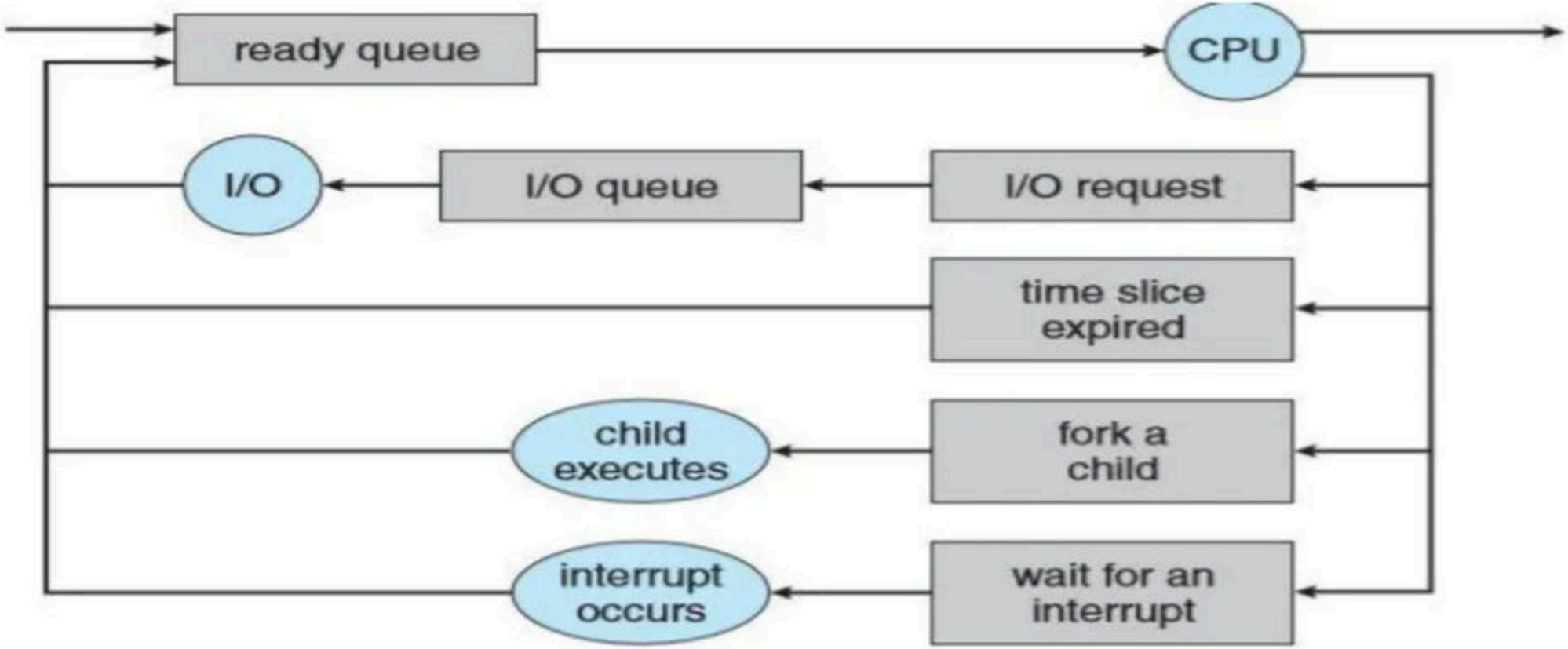


- The system also includes other queues. When a process is allocated the CPU, it executes for a while and eventually quits, is interrupted, or waits for the occurrence of a particular event, such as the completion of an I/O request.

- Suppose the process makes an I/O request to a shared device, such as a disk. Since there are many processes in the system, the disk may be busy with the I/O request of some other process. The process therefore may have to wait for the disk. The list of processes waiting for a particular I/O device is called a device queue. Each device has its own device queue.

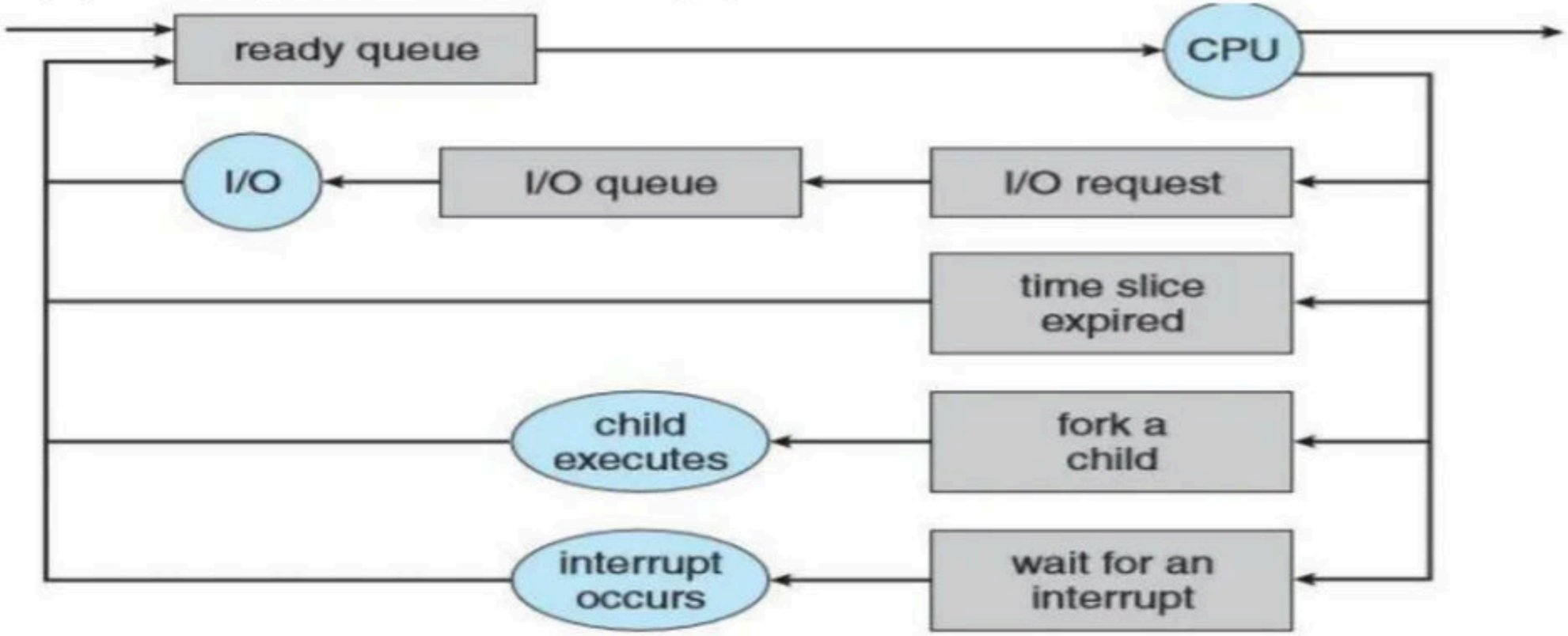


- Each rectangular box represents a queue. Two types of queues are present: the ready queue and a set of device queues. The circles represent the resources that serve the queues. A new process is initially put in the ready queue. It waits there until it is selected for execution, or dispatched.



Queueing-diagram representation of process scheduling.

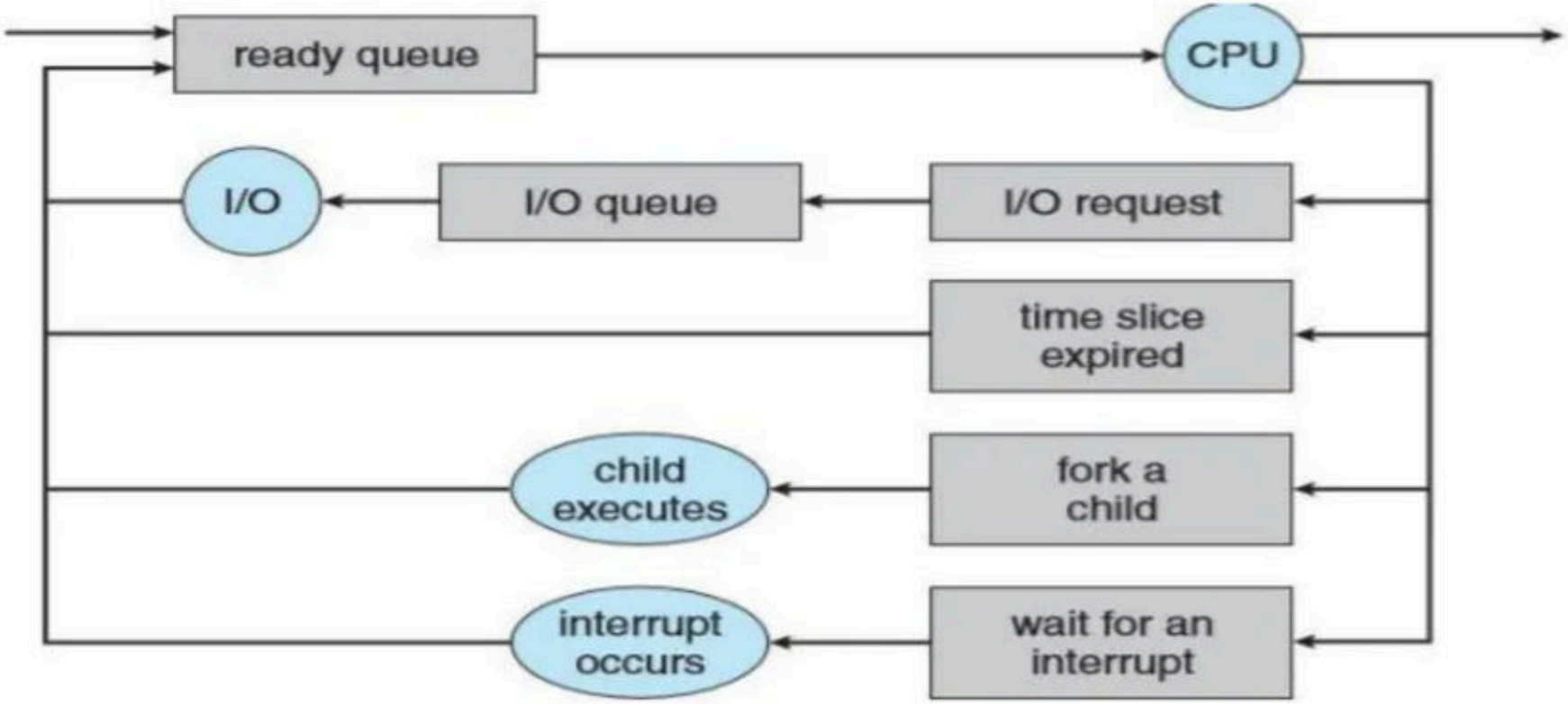
- Once the process is allocated the CPU and is executing, one of several events could occur:
  - The process could issue an I/O request and then be placed in an I/O queue.
  - The process could create a new child process and wait for the child's termination.
  - Time slice expired, the process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the ready queue.



Queueing-diagram representation of process scheduling.

- In the first two cases the process eventually switches from the waiting state to the ready state and is then put back in the ready queue.

- A process continues this cycle until it terminates, at which time it is removed from all queues and has its PCB and resources deallocated.



Queueing-diagram representation of process scheduling.

Break

Schedulers

- **Schedulers**: A process migrates among the various scheduling queues throughout its lifetime. The operating system must select, for scheduling purposes, processes from these queues in some fashion. The selection process is carried out by the appropriate scheduler.

- **Types of Schedulers**
  - **Long Term Schedulers (LTS)/Spooler:** In multiprogramming os, more processes are submitted than can be executed immediately. These processes are spooled to a mass-storage device (typically a disk), where they are kept for later execution. The long-term scheduler, or job scheduler, selects processes from this pool and loads them into memory for execution.

  - **Short Term Scheduler (STS):** The short-term scheduler, or CPU scheduler, selects from among the processes that are ready to execute and allocates the CPU to one of them.

- **Difference between LTS and STS** - The primary distinction between these two schedulers lies in frequency of execution.

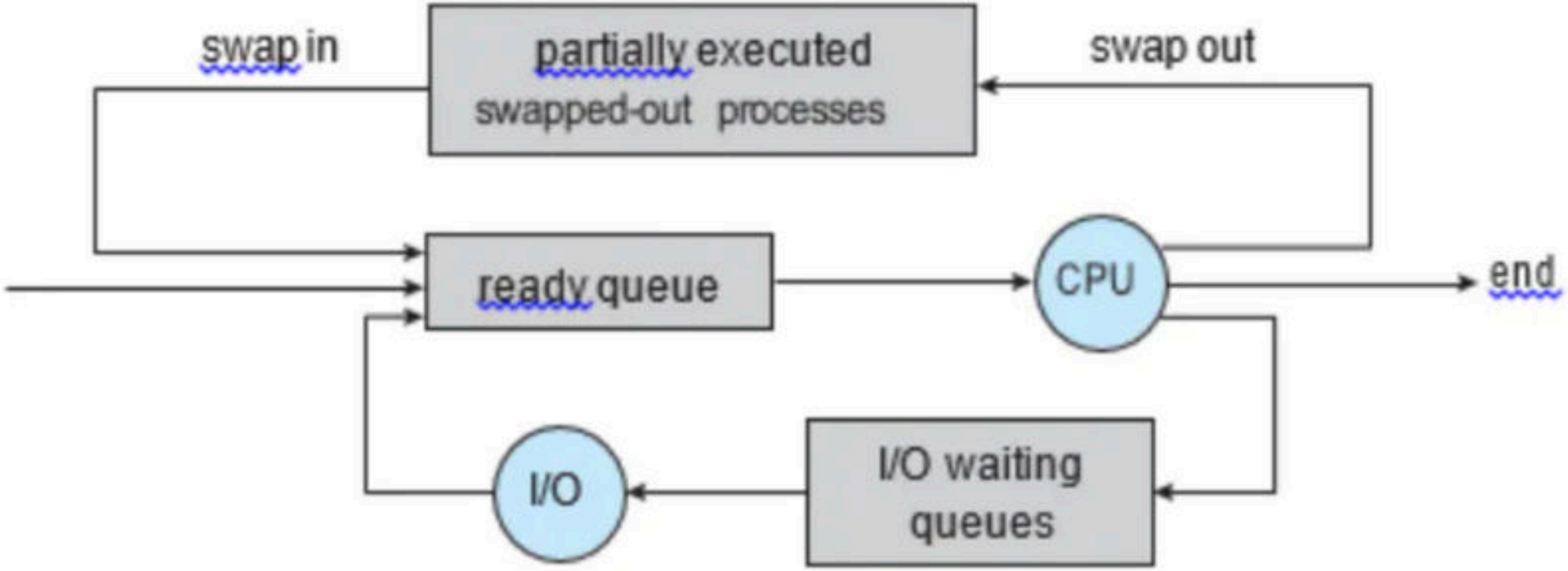
- The short-term scheduler must select a new process for the CPU frequently. A process may execute for only a few milliseconds before waiting for an I/O request. Often, the short-term scheduler executes at least once every 100 milliseconds.

- Because of the short time between executions, the short-term scheduler must be fast. The long-term scheduler on the other hand executes much less frequently; minutes may separate the creation of one new process and the next.

Degree of Multiprogramming - The number of processes in memory is known as Degree of Multiprogramming.

- The long-term scheduler controls the degree of multiprogramming as it is responsible for bringing in the processes to main memory.

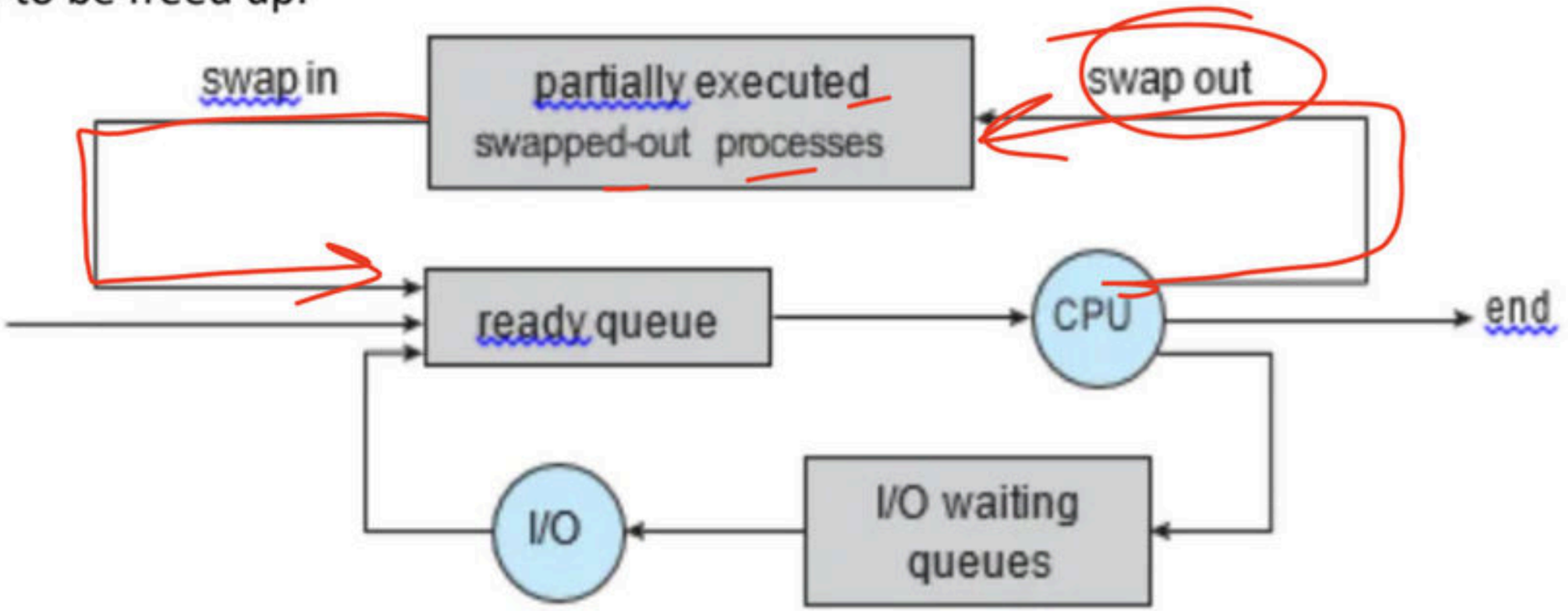
- If the degree of multiprogramming is stable, then the average rate of process creation must be equal to the average departure rate of processes leaving the system. So, this means the long-term scheduler may need to be invoked only when a process leaves the system. Because of the longer interval between executions, the long-term scheduler can afford to take more time to decide which process should be selected for execution.



Medium-term scheduler: The key idea behind a medium-term scheduler is that sometimes it can be advantageous to remove a process from memory (and from active contention for the CPU) and thus reduce the degree of multiprogramming.

Later, the process can be reintroduced into memory, and its execution can be continued where it left off. This scheme is called swapping.

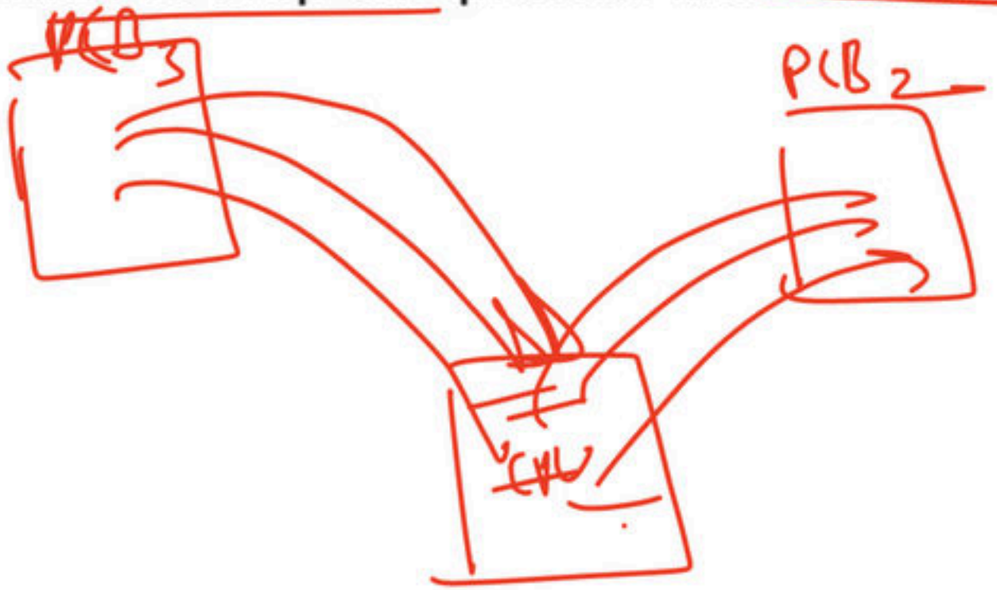
The process is swapped out, and is later swapped in, by the medium-term scheduler. Swapping may be necessary to improve the process mix or because a change in memory requirements has overcommitted available memory, requiring memory to be freed up.



- **Dispatcher** - The dispatcher is the module that gives control of the CPU to the process selected by the short-term scheduler.

- This function involves the following: Switching context, switching to user mode, jumping to the proper location in the user program to restart that program.

- The dispatcher should be as fast as possible, since it is invoked during every process switch. The time it takes for the dispatcher to stop one process and start another running is known as the dispatch latency.



Q A software to create a Job Queue is called..... **(NET-DEC-2006)**

a) Linkage editor

b) Interpreter

c) Driver

d) Spooler

Q A special software that is used to create a job queue is called **(NET-JUNE-2011)**

(A) Drive

(B) Spooler

(C) Interpreter

(D) Linkage editor

Q Which module gives control of the CPU to the process selected by the short - term scheduler? **(NET-NOV-2017)**

a) Dispatcher

b) Interrupt

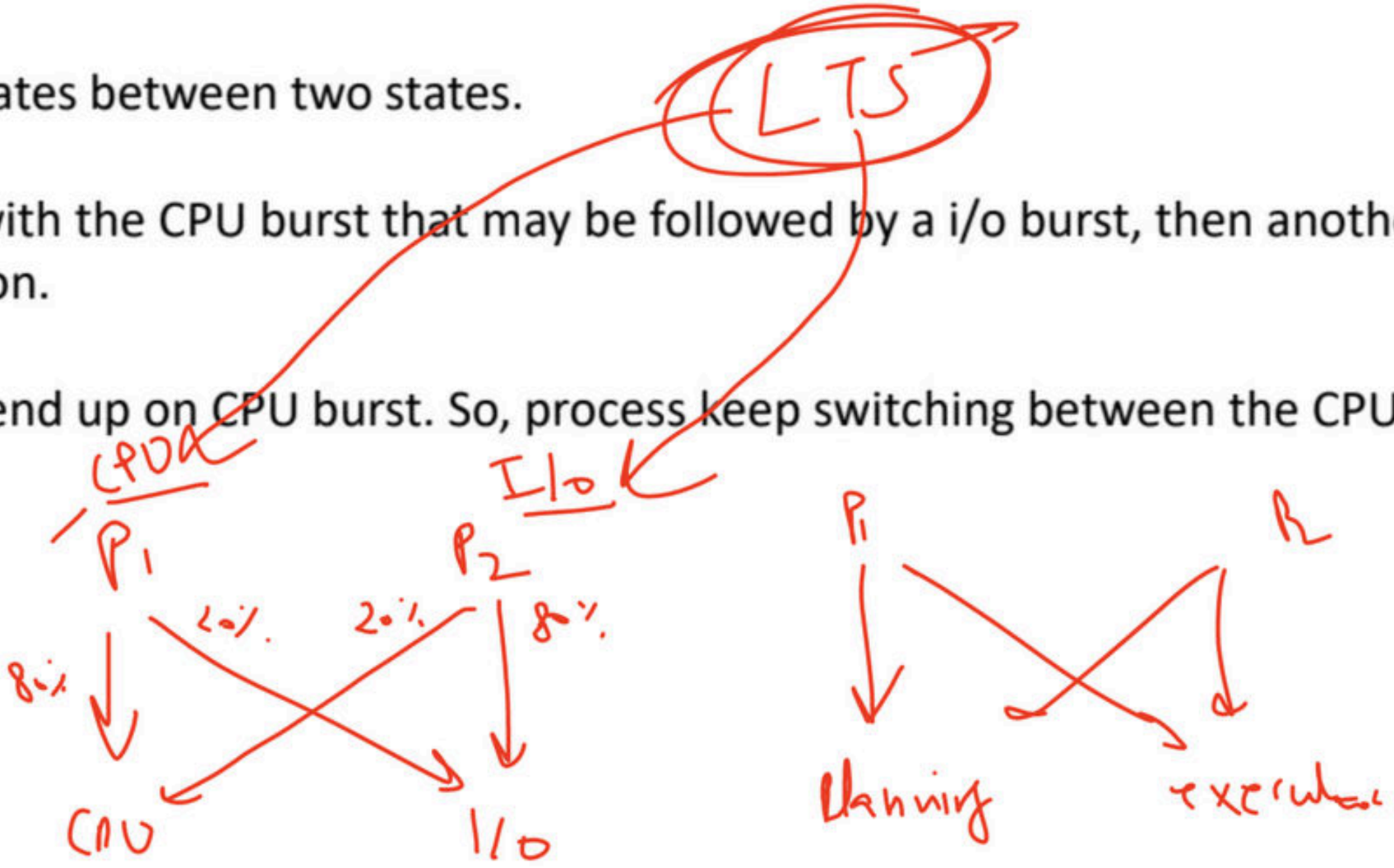
c) Scheduler

d) Threading

Break

CPU Bound and I/O Bound Processes

- A process execution consists of a cycle of CPU execution or wait and i/o execution or wait.
- Normally a process alternates between two states.
- Process execution begin with the CPU burst that may be followed by a i/o burst, then another CPU and i/o burst and so on.
- Eventually in the last will end up on CPU burst. So, process keep switching between the CPU and i/o during execution.



- **I/O Bound Processes:** An I/O-bound process is one that spends more of its time doing I/O than it spends doing computations.

- **CPU Bound Processes:** A CPU-bound process, generates I/O requests infrequently, using more of its time doing computations.

- It is important that the long-term scheduler select a good process mix of I/O-bound and CPU-bound processes.

- If all processes are I/O bound, the ready queue will almost always be empty, and the short-term scheduler will have little to do.

- Similarly, if all processes are CPU bound, the I/O waiting queue will almost always be empty, devices will go unused, and again the system will be unbalanced.

- So, to have the best system performance LTS needs to select a good combination of I/O and CPU Bound processes.

Break

Context Switch

- When an interrupt occurs, the system needs to save the current context of the process running on the CPU so that it can restore that context when its processing is done.

- Switching the CPU to another process requires performing a state save of the current process and a state restore of a different process. This task is known as a context switch. When a context switch occurs, the kernel saves the context of the old process in its PCB and loads the saved context of the new process scheduled to run. Context-switch time is pure overhead, because the system does no useful work while switching.

- Switching speed varies from machine to machine, depending on the memory speed, the number of registers that must be copied, and the existence of special instructions (such as a single instruction to load or store all registers). A typical speed is a few milliseconds.

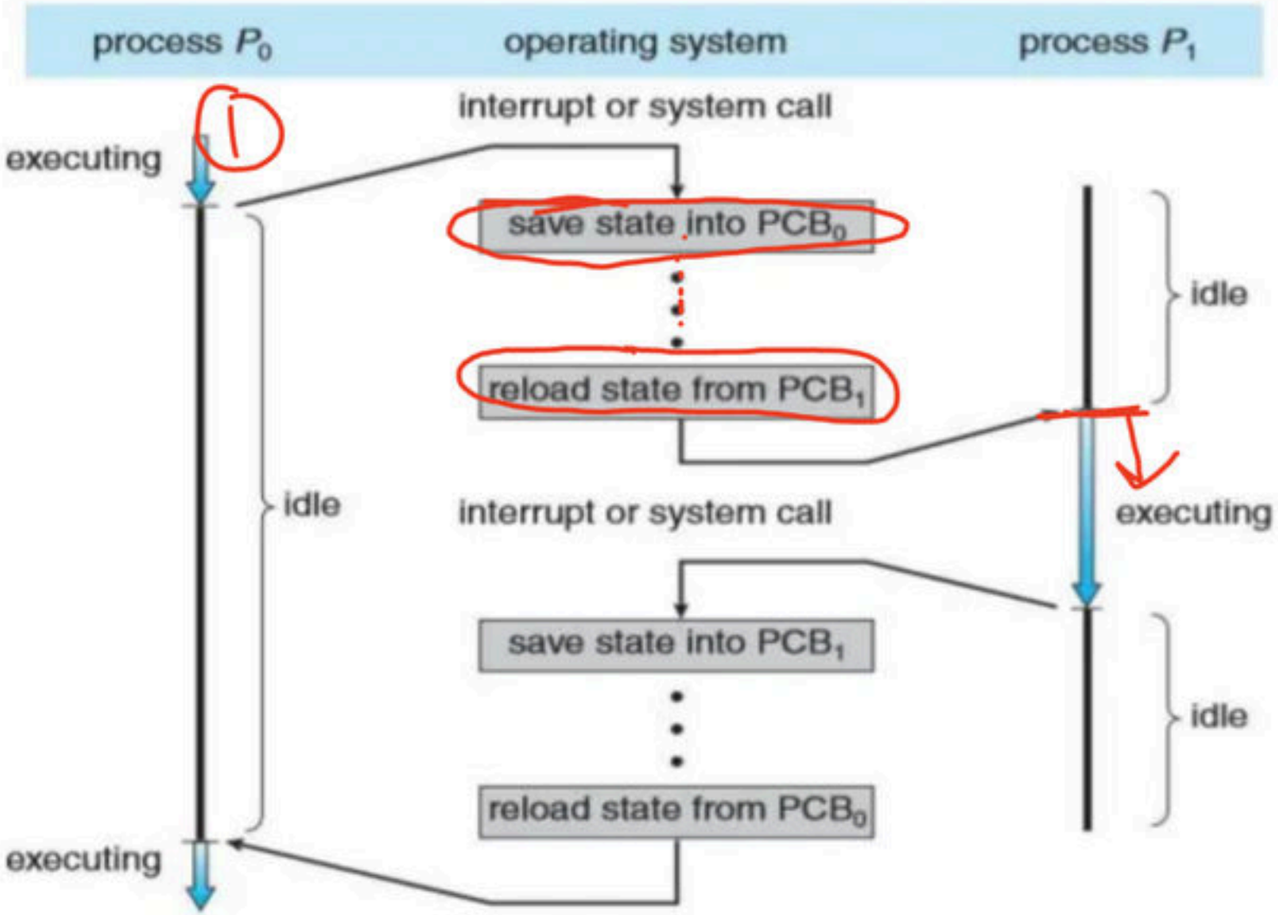


Diagram showing CPU switch from process to process.

Q Which of the following does not interrupt a running process? **(GATE-2001) (2 Marks)**

(A) A device

(B) Timer

(C) Scheduler process

(D) Power failure

Q Which is the correct definition of a valid process transition in an operating system?

a) Wake up: ready → running

b) Dispatch: ready → running

c) Block: ready → running

d) Timer runout: ready → running

Q A process stack does not contain

a) function parameters

b) local variables

c) return addresses

d) PID of child process

Q Loading operating system from secondary memory to primary memory is
called..... **(NET-DEC-2006)**

(A) Compiling

(B) Booting

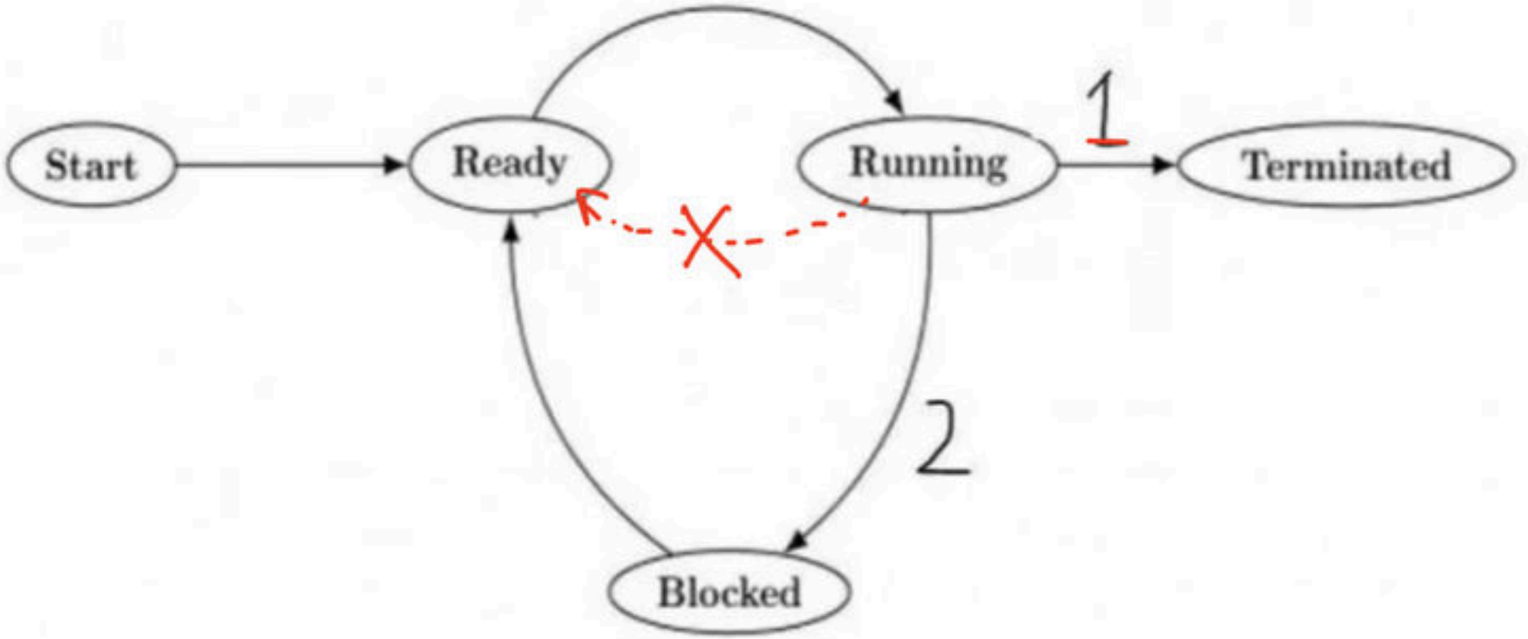
(C) Refreshing

(D) Reassembling

Type of scheduling

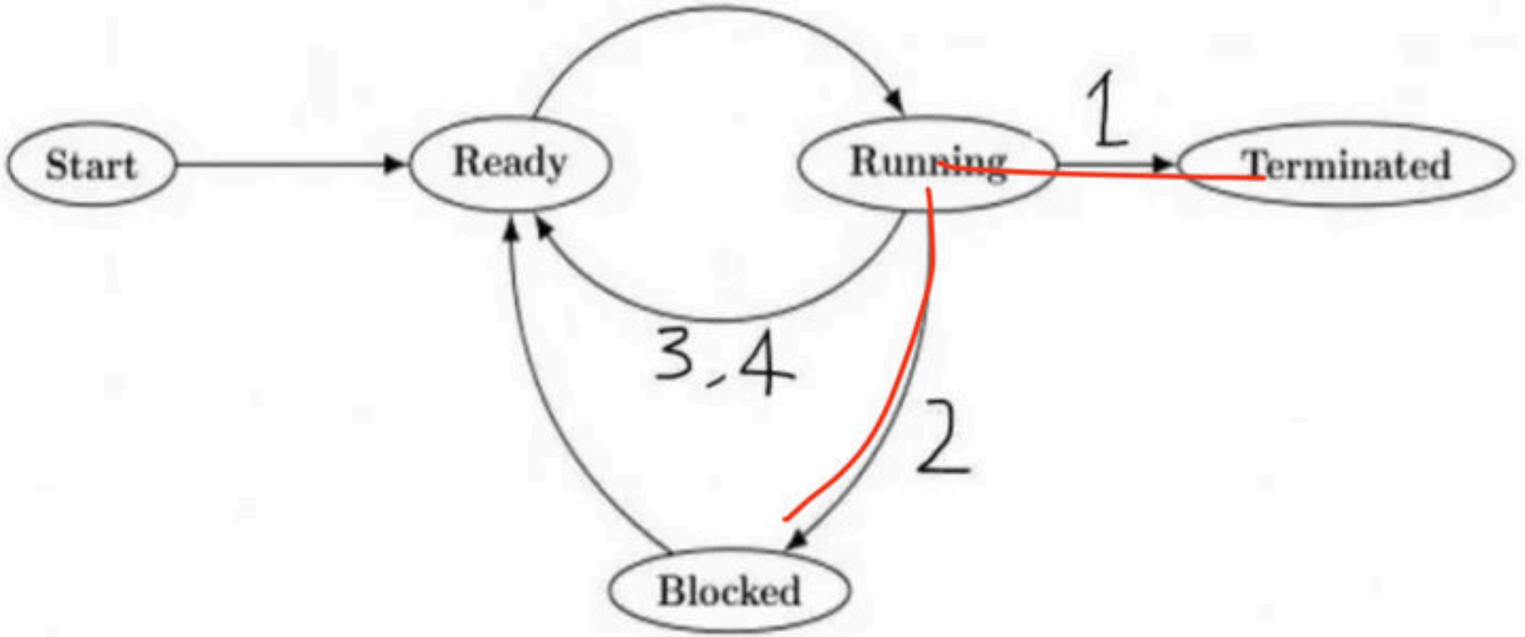
- **Non-Pre-emptive**: Under Non-Pre-emptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU willingly.

- A process will leave the CPU only
    1. When a process completes its execution (Termination state)
    2. When a process wants to perform some i/o operations(Blocked state)



Pre-emptive

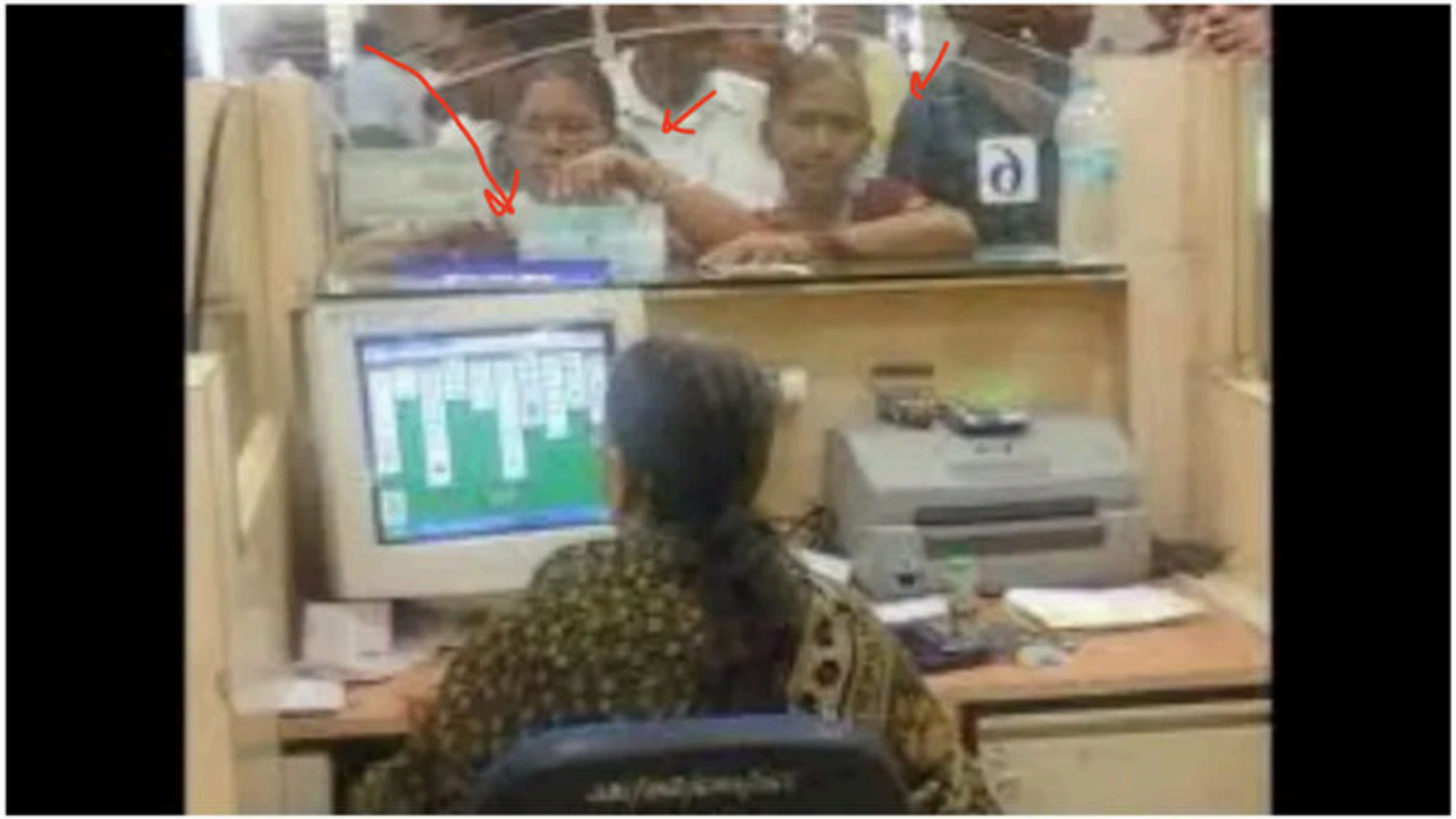
- Under Pre-emptive scheduling, once the CPU has been allocated to a process, A process will leave the CPU willingly or it can be forced out. So it will leave the CPU
  1. When a process completes its execution
  2. When a process leaves CPU voluntarily to perform some i/o operations
  3. If a new process enters in the ready states (new, waiting), in case of high priority
  4. When process switches from running to ready state because of time quantum expire.



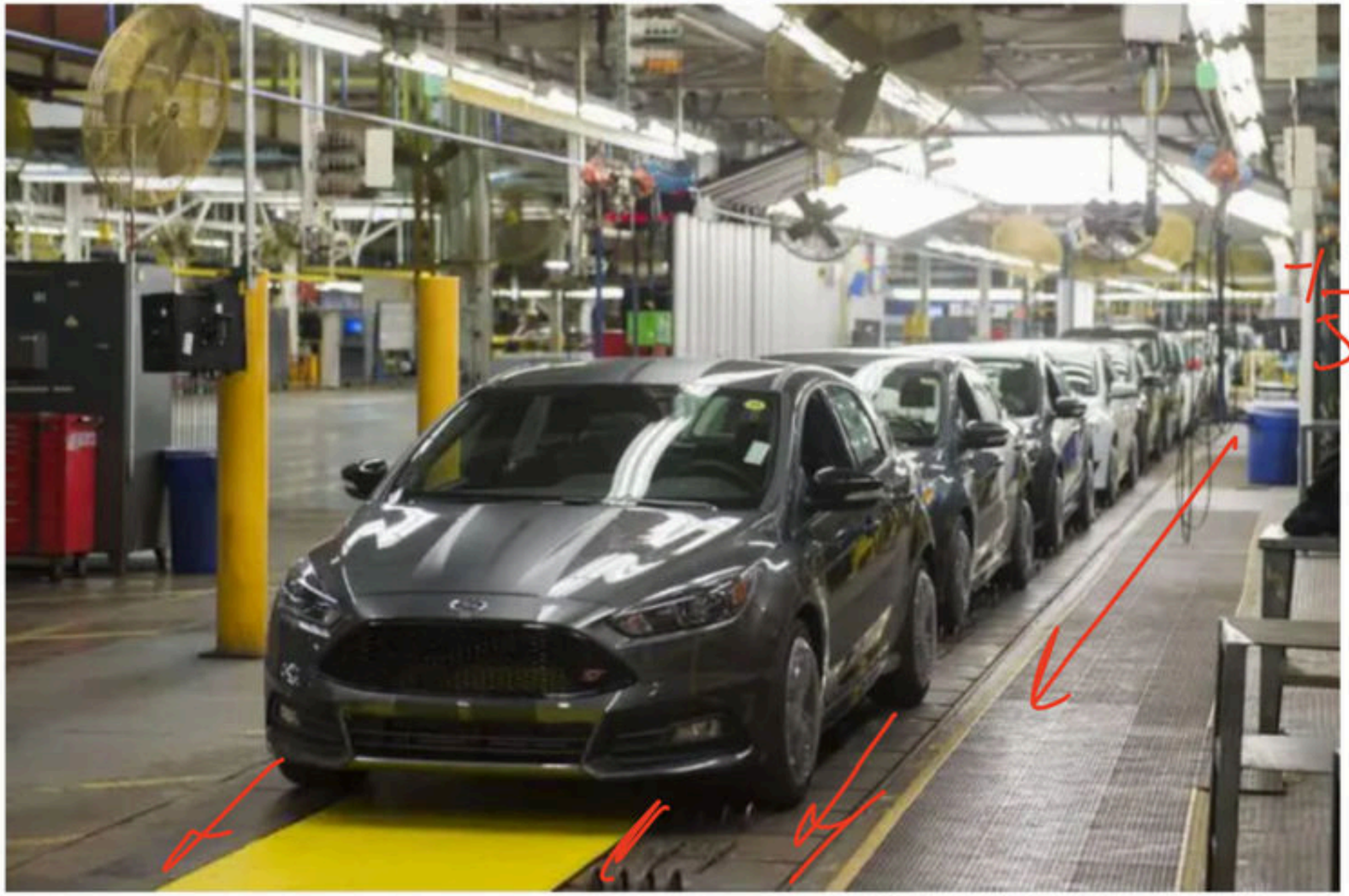
Break

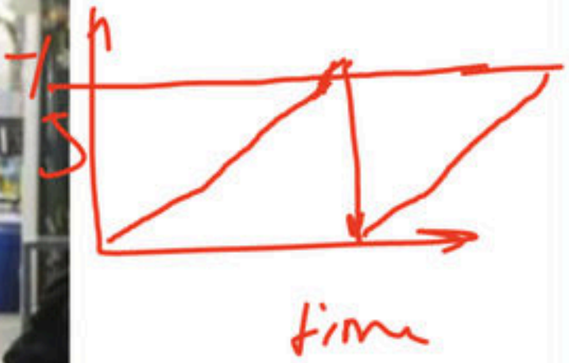
- **Scheduling criteria** - Different CPU-scheduling algorithms have different properties, and the choice of a particular algorithm may favour one class of processes over another. So, in order to efficiently select the scheduling algorithms following criteria should be taken into consideration:

- **CPU utilization:** Keeping the CPU as busy as possible.



- **Throughput:** If the CPU is busy executing processes, then work is being done. One measure of work is the number of processes that are completed per time unit, called throughput.





- **Waiting time:** Waiting time is the sum of the periods spent waiting in the ready queue.

- **Response Time:** Is the time it takes to start responding, not the time it takes to output the response.

- Note: The CPU-scheduling algorithm does not affect the amount of time during which a process executes I/0; it affects only the amount of time that a process spends waiting in the ready queue.

- It is desirable to maximize CPU utilization and throughput and to minimize turnaround time, waiting time, and response time.

Break

Terminology

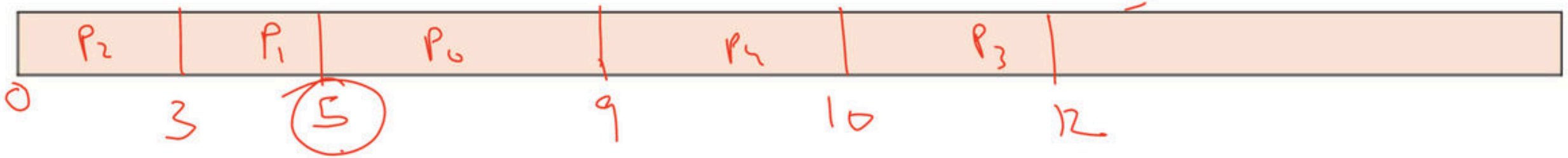
- **Arrival Time (AT):** Time at which process enters a ready state.
- **Burst Time (BT):** Amount of CPU time required by the process to finish its execution.
- **Completion Time (CT):** Time at which process finishes its execution.
- **Turn Around Time (TAT):** Completion Time (CT) – Arrival Time (AT), Waiting Time + Burst Time (BT)
- **Waiting Time:** Turn Around Time (TAT) – Burst Time (BT)

Break

FCFS (FISRT COME FIRST SERVE)

- FCFS is the simplest scheduling algorithm, as the name suggest, the process that requests the CPU first is allocated the CPU first.
- Implementation is managed by FIFO Queue.
- It is always non pre-emptive in nature.

P. No	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
P_0	2	4	9	9 - 2 = 7	7 - 4 = 3
P_1	1	2	5	5 - 1 = 4	4 - 2 = 2
P_2	0	3	3	3 - 0 = 3	3 - 3 = 0
P_3	4	2	12	12 - 4 = 8	8 - 2 = 6
P_4	3	1	10	10 - 3 = 7	7 - 1 = 6
Average				$\frac{7+4+3+8+7}{5} = 5.8$	$\frac{3+2+0+6+6}{5} = 3.4$



P. No	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
P_0	0	3	3	3 - 0 = 3	3 - 3 = 0
P_1	2	2	5	5 - 2 = 3	3 - 2 = 1
P_2	6	4	10	10 - 6 = 4	4 - 4 = 0
Average				$\frac{3+3+4}{3}$	$\frac{0+1+0}{3}$

99.99%

0.01%

1 Gbps

Break

P. No	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
P_0	6	**4**			
P_1	2	**5**			
P_2	3	**3**			
P_3	1	**1**			
P_4	4	**2**			
P_5	5	**6**			
Average					

Q Consider the following table of arrival time and burst time for three processes P_0, P_1, P_2, P_3, P_4 What is the average Waiting Time and Turnaround Time for the five processes?

P. No	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
P_0	0	14			
P_1	1	3			
P_2	2	1			
P_3	3	2			
P_4	4	5			
Average					

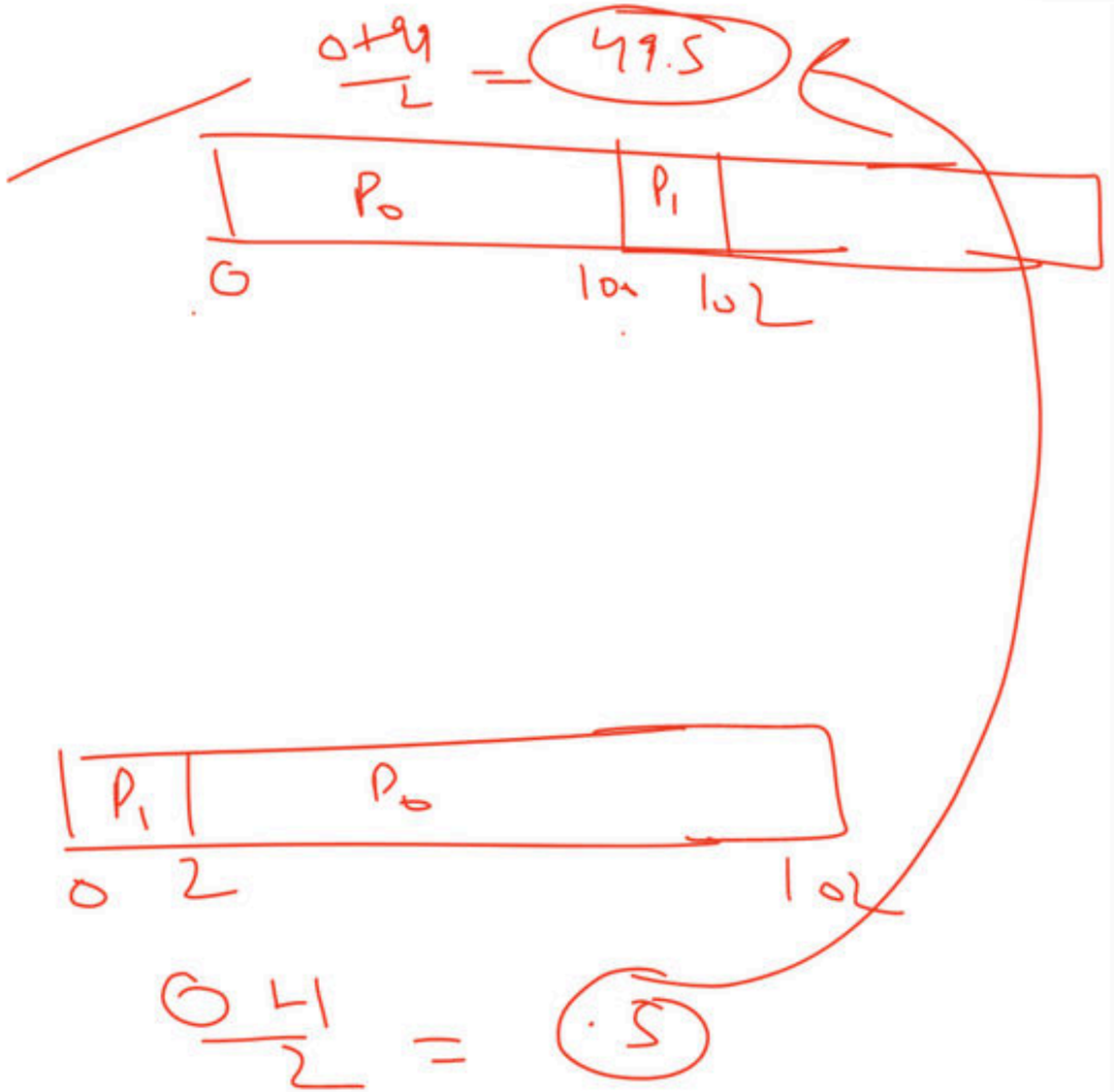
Break

Advantage

- Easy to understand, and can easily be implemented using Queue data structure.
- Can be used for Background processes where execution is not urgent.

P. No	AT	BT	TAT=CT-AT	WT=TAT -BT
P_0	0	100		0
P_1	1	2		99
Average				

P. No	AT	BT	TAT=CT-AT	WT=TAT -BT
P_0	1	100		
P_1	0	2		
Average				



Convoy Effect

- If the smaller process have to wait more for the CPU because of Larger process then this effect is called Convoy Effect, it result into more average waiting time.

- Solution, smaller process have to be executed before longer process, to achieve less average waiting time.

Disadvantage

- FCFS suffers from convoy which means smaller process have to wait larger process, which result into large average waiting time.

- The FCFS algorithm is thus particularly troublesome for time-sharing systems (due to its non-pre-emptive nature), where it is important that each user get a share of the CPU at regular intervals.

- Higher average waiting time and TAT compared to other algorithms.

Break

Shortest Job First (SJF)(non-pre-emptive)

Shortest Remaining Time First (SRTF)/ (Shortest Next CPU Burst) (Pre-emptive)

- Whenever we make a decision of selecting the next process for CPU execution, out of all available process, CPU is assigned to the process having smallest burst time requirement. When the CPU is available, it is assigned to the process that has the smallest next CPU burst. If there is a tie, FCFS is used to break tie.

- It supports both version non-pre-emptive and pre-emptive (purely greedy approach)

- In Shortest Job First (SJF)(non-pre-emptive) once a decision is made and among the available process, the process with the smallest CPU burst is scheduled on the CPU, it cannot be pre-empted even if a new process with the smaller CPU burst requirement then the remaining CPU burst of the running process enter in the system.

- In Shortest Remaining Time First (SRTF) (Pre-emptive) whenever a process enters in ready state, again we make a scheduling decision weather, this new process with the smaller CPU burst requirement then the remaining CPU burst of the running process and if it is the case then the running process is pre-empted and new process is scheduled on the CPU.

- This version (SRTF) is also called optimal is it guarantee minimal average waiting time.

P. No	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
P_0	1	7			
P_1	2	5			
P_2	3	1			
P_3	4	2			
P_4	5	8			
Average					

P. No	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
P_0	0	6			
P_1	1	4			
P_2	2	3			
P_3	3	1			
P_4	4	2			
P_5	5	1			
Average					

P. No	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
P_0	3	4			
P_1	4	2			
P_2	5	1			
P_3	2	6			
P_4	1	8			
P_5	2	4			
Average					

Break

Q Consider an arbitrary set of CPU-bound processes with unequal CPU burst lengths submitted at the same time to a computer system. Which one of the following process scheduling algorithms would minimize the average waiting time in the ready queue? **(GATE - 2016) (1 Marks)**

(a) Shortest remaining time first

(b) Round-robin with time quantum less than the shortest CPU burst

(c) Uniform random

(d) Highest priority first with priority proportional to CPU burst length

Q For the processes listed in the following table, which of the following scheduling schemes will give the lowest average turnaround time? **(GATE-2015) (2 Marks)**

Process	Arrival Time	Processing Time
A	0	3
B	1	6
C	4	4
D	6	2

(A) First Come First Serve
(B) Non-pre-emptive Shortest Job First
(C) Shortest Remaining Time
(D) Round Robin with Quantum value two

Q Consider the following four processes with arrival times (in milliseconds) and their length of CPU bursts (in milliseconds) as shown below: **(GATE - 2019) (2 Marks)**

Process	Arrival Time	CPU Time	CT	TAT	WT
P_1	0	3			
P_2	1	1			
P_3	3	3			
P_4	4	Z			

These processes are run on a single processor using pre-emptive Shortest Remaining Time First scheduling algorithm. If the average waiting time of the processes is 1 millisecond, then the value of Z is _____.

Break

Advantage

- Pre-emptive version guarantees minimal average waiting time so some time also referred as optimal algorithm.
- Provide a standard for other algo in terms of average waiting time
- Provide better average response time compare to FCFS

Disadvantage

- This algo cannot be implemented as there is no way to know the length of the next CPU burst.
- Here process with the longer CPU burst requirement goes into starvation.
- No idea of priority, longer process has poor response time.

Break

Q Consider the following three processes with the arrival time and CPU burst time given in milliseconds: **(NET-July-2018)**

Process	Arrival Time	Burst Time
P_1	0	7
P_2	1	4
P_3	2	8

The Gantt Chart for pre-emptive SJF scheduling algorithm is _____.

a)

P_1(0-7)	P_2(7-13)	P_3(13-21)

b)

P_1(0-1)	P_2(1-5)	P_1(5-11)	P_3(11-19)

c)

P_1(0-7)	P_2(7-11)	P_3(11-19)

d)

P_2(0-4)	P_3(4-12)	P_1(12-19)

Q Consider the following four processes with the arrival time and length of CPU burst given in milliseconds: **(NET-Nov-2018)**

Process	Arrival Time	Burst Time	CT	TAT	WT
P_1	0	8			
P_2	1	4			
P_3	2	9			
P_4	3	5			

The average waiting time for pre-emptive SJF scheduling algorithm is _____.

a) 6.5 **b)** 7.5 **c)** 6.75 **d)** 7.75

Q Consider the following set of processes, with the arrival times and the CPU-burst times given in milliseconds **(GATE - 2017) (2 Marks)**

Process	Arrival Time	CPU Time	CT	TAT	WT
P_1	0	5			
P_2	1	3			
P_3	2	3			
P_4	4	1			

What is the average turnaround time for these processes with the pre-emptive shortest remaining processing time first (SRPT) algorithm?

(A) 5.50 **(B)** 5.75 **(C)** 6.00 **(D)** 6.25

Break

Q Consider the following CPU processes with arrival times (in milliseconds) and length of CPU bursts (in milliseconds) as given below: **(GATE - 2017) (1 Marks)**

Process	Arrival Time	CPU Time	CT	TAT	WT
P_1	0	7			
P_2	3	3			
P_3	5	5			
P_4	6	2			

If the pre-emptive shortest remaining time first scheduling algorithm is used to schedule the processes, then the average waiting time across all processes is _____ milliseconds.

Q Consider three CPU intensive processes P_1, P_2, P_3 which require 20,10 and 30 units of time, arrive at times 1,3 and 7 respectively. Suppose operating system is implementing Shortest Remaining Time first (pre-emptive scheduling) algorithm, then _____ context switches are required (suppose context switch at the beginning of Ready queue and at the end of Ready queue are not counted). **(NET-Aug-2016)**

Process	Arrival Time	Burst Time
P_1	1	20
P_2	3	10
P_3	7	30

a) 3 **b)** 2 **c)** 4 **d)** 5

Q Consider the following processes, with the arrival time and the length of the CPU burst given in milliseconds. The scheduling algorithm used is pre-emptive shortest remaining-time first.

Process	Arrival Time	CPU Time	CT	TAT	WT
P_1	0	10			
P_2	3	6			
P_3	7	1			
P_4	8	3			

The average turnaround time of these processes is _____ milliseconds. **(GATE - 2016) (2 Marks)**

Break

Q Consider the following set of processes that need to be scheduled on a single CPU. All the times are given in milliseconds? **(GATE-2014) (2 Marks)**

Process Name	Arrival Time	Execution Time	CT	TAT	WT
A	0	6			
B	3	2			
C	5	4			
D	7	6			
E	10	3			

Using the *shortest remaining time first* scheduling algorithm, the average process turnaround time (in msec) is _____.

Q An operating system uses shortest remaining time first scheduling algorithm for pre-emptive scheduling of processes. Consider the following set of processes with their arrival times and CPU burst times (in milliseconds) **(GATE-2014) (2 Marks)**

Process	Arrival Time	Burst Time	CT	TAT	WT
P_1	0	12			
P_2	2	4			
P_3	3	6			
P_4	8	5			

The average waiting time (in milliseconds) of the processes is _____.

Q Consider the following table of arrival time and burst time for three processes P_0, P_1 and P_2.

Process	Arrival Time	Burst Time	CT	TAT	WT
P_0	0	9			
P_1	1	4			
P_2	2	9			

The pre-emptive shortest job first scheduling algorithm is used. Scheduling is carried out only at arrival or completion of processes. What is the average waiting time for the three processes?

(GATE-2011) (2 Marks)

(A) 5.0 ms **(B)** 4.33 ms **(C)** 6.33 **(D)** 7.33

Break

Q An operating system uses Shortest Remaining Time first (SRT) process scheduling algorithm. Consider the arrival times and execution times for the following processes:

Process	Arrival Time	CPU Time	CT	TAT	WT
P_1	0	20			
P_2	15	25			
P_3	30	10			
P_4	45	15			

What is the total waiting time for process P_2? **(GATE - 2007) (2 Marks)**

(A) 5 **(B)** 15 **(C)** 40 **(D)** 55

Q Consider three CPU-intensive processes, which require 10, 20 and 30 time units and arrive at times 0, 2 and 6, respectively. How many context switches are needed if the operating system implements a shortest remaining time first scheduling algorithm? Do not count the context switches at time zero and at the end? **(GATE-2006) (1 Marks)**

Process	Arrival Time	CPU Time
P_1	0	10
P_2	2	20
P_3	6	30

(A) 1 **(B)** 2 **(C)** 3 **(D)** 4

Break

- As SJF is not implementable, we can use the one technique where we try to predict the CPU burst of the next coming process.

- The method is used as exponential averaging technique, where we consider the previous value and previous prediction.

$$Tau_{(n+1)} = alpa\ t_n + (1- alpa)\ tau_{n.}$$

Process	t	tau
P_1	10	20
P_2	12	
P_3	14	
P_4		

$$Tau_{(n+1)} = alpa\ t_n + (1- alpa)\ tau_n$$

- This idea is also more of theoretical importance as most of the time the burst requirement of the coming process may vary by a large extent and if the burst time requirement of all the process is approximately same then there is no advantage of using this scheme.

Break

Priority scheduling

- Here a priority is associated with each process. At any instance of time out of all available process, CPU is allocated to the process which possess highest priority (may be higher or lower number).

- Tie is broken using FCFS order. No importance to senior or burst time. It supports both non-pre-emptive and pre-emptive versions.

- In Priority (non-pre-emptive) once a decision is made and among the available process, the process with the highest priority is scheduled on the CPU, it cannot be pre-empted even if a new process with higher priority more than the priority of the running process enter in the system.

- In Priority (pre-emptive) once a decision is made and among the available process, the process with the highest priority is scheduled on the CPU.

- if it a new process with priority more than the priority of the running process enter in the system, then we do a context switch and the processor is provided to the new process with higher priority.

- Priorities are generally indicated by some fixed range of numbers, such as 0 to 7 or 0 to 4,095. There is no general agreement on whether 0 is the highest or lowest priority, it can vary from systems to systems.

P. No	AT	BT	Priority	CT	TAT = CT - AT	WT = TAT - BT
P_0	1	4	4			
P_1	2	2	5			
P_2	2	3	7			
P_3	3	5	8(H)			
P_4	3	1	5			
P_5	4	2	6			
Average						

P. No	AT	BT	Priority	CT	TAT = CT - AT	WT = TAT - BT
P_0	0	50	4			
P_1	20	20	1(h)			
P_2	40	100	3			
P_3	60	40	2			
Average						

P. No	AT	BT	Priority	CT	TAT = CT - AT	WT = TAT - BT
P_0	1	4	5			
P_1	2	5	2			
P_2	3	6	6			
P_3	0	1	4			
P_4	4	2	1			
P_5	5	3	8(H)			
Average						

Break

- **Advantage**
  - Gives a facility specially to system process.
  - Allow us to run important process even if it is a user process.

- **Disadvantage**
  - Here process with the smaller priority may starve for the CPU
  - No idea of response time or waiting time.

- Note: - Specially use to support system process or important user process

- **Ageing**: - a technique of gradually increasing the priority of processes that wait in the system for long time. E.g. priority will increase after every 10 mins

Break

Q Consider the set of processes with arrival time (in milliseconds), CPU burst time (in milliseconds), and priority (0 is the highest priority) shown below. None of the processes have I/O burst time.

Process	Arrival Time	Burst Time	Priority	CT	TAT	WT
P_1	0	11	2			
P_2	5	28	0			
P_3	12	2	3			
P_4	2	10	1			
P_5	9	16	4			

The average waiting time (in milliseconds) of all the processes using preemptive priority scheduling algorithm is _____. **(GATE-2017) (2 Marks)**

Q Some of the criteria for calculation of priority of a process are:

a. Processor utilization by an individual process.

b. Weight assigned to a user or group of users.

c. Processor utilization by a user or group of processes

In fair share scheduler, priority is calculated based on: **(NET-Jan-2017)**

a) only (a) and (b)

b) only (a) and (c)

c) (a), (b) and (c)

d) only (b) and (c)

Break

Q Consider a uniprocessor system executing three tasks T_1, T_2 and T_3, each of which is composed of an infinite sequence of jobs (or instances) which arrive periodically at intervals of 3, 7 and 20 milliseconds, respectively. The priority of each task is the inverse of its period and the available tasks are scheduled in order of priority, with the highest priority task scheduled first. Each instance of T_1, T_2 and T_3 requires an execution time of 1, 2 and 4 milliseconds, respectively. Given that all tasks initially arrive at the beginning of the 1st milliseconds and task preemptions are allowed, the first instance of T_3 completes its execution at the end of _____ milliseconds. **(GATE-2015) (2 Marks)**

Process	Arrival Time	Burst Time

Break

Q The problem of indefinite blockage of low-priority jobs in general priority scheduling algorithm can be solved using: **(NET-Dec-2012)**

a) Parity bit **b)** Aging **c)** Compaction **d)** Timer

Q We wish to schedule three processes P_1, P_2 and P_3 on a uniprocessor system. The priorities, CPU time requirements and arrival times of the processes are as shown below.

Process	Priority	CPU time	Arrival time
P_1	10(highest)	20 sec	00:00:05
P_2	9	10 sec	00:00:03
P_3	8 (lowest)	15 sec	00:00:00

We have a choice of pre-emptive or non-pre-emptive scheduling. In pre-emptive scheduling, a late-arriving higher priority process can pre-empt a currently running process with lower priority. In non-pre-emptive scheduling, a late-arriving higher priority process must wait for the currently executing process to complete before it can be scheduled on the processor. What are the turnaround times (time from arrival till completion) of P_2 using pre-emptive and non-pre-emptive scheduling respectively. **(GATE-2005) (2 Marks)**

(A) 30 sec, 30 sec **(B)** 30 sec, 10 sec **(C)** 42 sec, 42 sec **(D)** 30 sec, 42 sec

Break

Round robin

Round robin

- This algo is designed for time sharing systems, where it is not, necessary to complete one process and then start another, but to be responsive and divide time of CPU among the process in the ready state. Here ready queue is treated as a circular queue (FIFO).

- It is similar to FCFS scheduling, but pre-emption is added to enable the system to switch between processes.

- The ready queue is treated as a circular queue. The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval equivalent 1 Time quantum (where value of TQ can be anything).

- We fix a time quantum, up to which a process can hold the CPU in one go, with in which either a process terminates or process must release the CPU and enter the ready queue and wait for the next chance.

- The process may have a CPU burst of less than given time quantum. In this case, the process itself will release the CPU voluntarily.

- CPU Scheduler will select the next process for execution. OR The CPU burst of the currently running process is longer than 1-time quantum, the timer will go off and will cause an interrupt to the operating system.

- A context switch will be executed, and the process will be put at the tail of the ready queue. The CPU scheduler will then select the next process in the ready queue.

- If there are n processes in the ready queue and the time quantum is q, then each process gets 1/n of the CPU time in chunks of at most q time units.

- Each process must wait no longer than (n - 1) x q time units until its next time quantum.

P. No	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
P_0	0	4			
P_1	1	5			
P_2	2	2			
P_3	3	1			
P_4	4	6			
P_5	6	3			
Average					

P. No	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
P_0	5	5			
P_1	4	6			
P_2	3	7			
P_3	1	9			
P_4	2	2			
P_5	6	3			
Average					

- If the time quantum is extremely large, the RR policy is the same as the FCFS policy.

- If the time quantum is extremely small (say, 1 millisecond), the RR approach is called processor sharing and (in theory) creates the appearance that each of n processes has its own processor running at 1/n the speed of the real processor. We also need also to consider the effect of context switching on the performance of RR scheduling.

- **Advantage**
  - Perform best in terms of average response time
  - Works will in case of time-sharing systems, client server architecture and interactive system
  - kind of SJF implementation

- **Disadvantage**
  - Longer process may starve
  - Performance depends heavily on time quantum - If value of the time quantum is very less, then it will give lesser average response time (good but total no of context switches will be more, so CPU utilization will be less), If time quantum is very large then average response time will be more bad, but no of context switches will be less, so CPU utilization will be good.
  - No idea of priority

Break

Q consider the following set of processes and the length of CPU burst time given in milliseconds:

Process	CPU Burst Time	CT	TAT	WT
P_1	5			
P_2	7			
P_3	6			
P_4	4			

Assume that processes being scheduled with round robin scheduling algorithm with time quantum 4ms. Then the waiting for p4 is _____. **(NET-DEC-2018)**

a) 0 **b) 4** **c) 6** **d) 12**

Q Consider the following set of processes with the length of CPU burst time in milliseconds (ms):

Process	Burst Time	Priority	CT	TAT	WT
A	6	3			
B	1	1			
C	2	3			
D	1	4			
E	5	2			

Assume that processes are stored in ready queue in following order: A – B – C – D – E
Using round robin scheduling with time slice of 1 ms, the average turnaround time is **(NET-Spet-2013)**

(A) 8.4 ms **(B)** 12.4 ms **(C)** 9.2 ms **(D)** 9.4 ms

Q A scheduling algorithm assigns priority proportional to the waiting time of a process. Every process starts with priority zero (the lowest priority). The scheduler re-evaluates the process priorities every T time units and decides the next process to schedule. Which one of the following is TRUE if the processes have no I/O operations and all arrive at time zero? **(GATE-2013) (1 Marks)**

(A) This algorithm is equivalent to the first-come-first-serve algorithm

(B) This algorithm is equivalent to the round-robin algorithm.

(C) This algorithm is equivalent to the shortest-job-first algorithm..

(D) This algorithm is equivalent to the shortest-remaining-time-first algorithm

Break

Q In round robin CPU scheduling as time quantum is increased the average turnaround time **(NET-June-2012)**

(A) Increases

(B) Decreases

(C) remains constant

(D) varies irregularly

Q Consider n processes sharing the CPU in a round-robin fashion. Assuming that each process switch takes s seconds, what must be the quantum size q such that the overhead resulting from process switching is minimized but at the same time each process is guaranteed to get its turn at the CPU at least every t seconds? **(GATE-1998) (1 Marks) (NET-Dec-2012)**

a) $q \leq (t-ns)/(n-1)$

b) $q \geq (t-ns)/(n-1)$

c) $q \leq (t-ns)/(n+1)$

d) $q \geq (t-ns)/(n+1)$

Q If the time-slice used in the round-robin scheduling policy is more than the maximum time required to execute any process, then the policy will **(GATE-2008)** **(1 Marks)**

(A) degenerate to shortest job first

(B) degenerate to priority scheduling

(C) degenerate to first come first serve

(D) none of the above

Break

Q In processor management, round robin method essentially uses the pre-emptive version of ..... **(NET-JUNE-2006)**

(A) FILO **(B)** FIFO **(C)** SJF **(D)** Longest time first

Q Four jobs to be executed on a single processor system arrive at time 0^+ in the order A, B, C, D. their burst CPU time requirements are 4, 1, 8, 1 time units respectively. The completion time of A under round robin scheduling with time slice of one-time unit is. **(GATE-1996) (2 Marks) (ISRO-2008)**

(a) 10 **(b) 4** **(c) 8** **(d) 9**

Q Which scheduling policy is most suitable for a time-shared operating system?
(GATE-1995) (1 Marks)

(a) Shortest Job First

(b) Round Robin

(c) First Come First Serve

(d) Elevator

Q Assume that the following jobs are to be executed on a single processor system
The jobs are assumed to have arrived at time 0^+ and in the order p, q, r, s, t. calculate the departure time (completion time) for job p if scheduling is round robin with time slice 1. **(GATE-1993) (2 Marks)**

Job Id	CPU Burst Time
P	4
Q	1
R	8
S	1
T	2

Break

Q In which of the following scheduling criteria, context switching will never take place? **(NET-July-2018)**

a) Round Robin

b) Pre-emptive SJF

c) Non-Pre-emptive SJF

d) Preemptive priority

Q Which of the following scheduling algorithms may cause starvation? **(NET-Jan-2017)**

a. First-come-first-served **b.** Round Robin **c.** Priority

d. Shortest process next **e.** Shortest remaining time first

a) a, c and e

b) c, d and e

c) b, d and e

d) b, c and d

Q A scheduling Algorithm assigns priority proportional to the waiting time of a process. Every process starts with priority zero (lowest priority). The scheduler reevaluates the process priority for every 'T' time units and decides next process to be scheduled. If the process has no I/O operations and all arrive at time zero, then the scheduler implements _____ criteria. **(NET-June-2016)**

a) Priority scheduling

b) Round Robin Scheduling

c) Shortest Job First

d) FCFS

Q Three processes A, B and C each execute a loop of 100 iterations. In each iteration of the loop, a process performs a single computation that requires t_c CPU milliseconds and then initiates a single I/O operation that lasts for $t_{i/o}$ milliseconds. It is assumed that the computer where the processes execute has sufficient number of I/O devices and the OS of the computer assigns different I/O devices to each process. Also, the scheduling overhead of the OS is negligible. The processes have the following characteristics:

Process Id	T_c	$T_{i/o}$
A	100	500
B	350	500
C	200	500

The processes A, B, and C are started at times 0, 5 and 10 milliseconds respectively, in a pure time-sharing system (round robin scheduling) that uses a time slice of 50 milliseconds. The time in milliseconds at which process C would complete its first I/O operation is _____. **(GATE-2014) (2 Mark)**

Break

Q Consider the following processes with time slice of 4 milliseconds (I/O requests are ignored): The average turnaround time of these processes will be **(NET-June-2013)**

Process	Arrival Time	Processing Time	CT	TAT	WT
A	0	8			
B	1	4			
C	2	9			
D	3	5			

(A) 19.25 milliseconds
(B) 18.25 milliseconds
(C) 19.5 milliseconds
(D) 18.5 milliseconds

Q Pre-emptive scheduling is the strategy of temporarily suspending a running process **(NET-June-2012)**

a) before the CPU time slice expires

b) to allow starving processes to run

c) when it requires I/O

d) to avoid collision

Q Consider the 3 processes, P_1, P_2 and P_3 shown in the table. **(GATE-2012) (2 Marks)**

Process	Arrival Time	Time Unit Required
P_1	0	5
P_2	1	7
P_3	3	4

The completion order of the 3 processes under the policies FCFS and RR2 (round robin scheduling with CPU quantum of 2-time units) are

(A) FCFS: P_1, P_2, P_3 RR: P_1, P_2, P_3

(B) FCFS: P_1, P_3, P_2 RR: P_1, P_3, P_2

(C) FCFS: P_1, P_2, P_3 RR: P_1, P_3, P_2

(D) FCFS: P_1, P_3, P_2 RR: P_1, P_2, P_3

Break

Q Which of the following statements are true? **(GATE-2010) (2 Marks)**

1) Shortest remaining time first scheduling may cause starvation

2) Pre-emptive scheduling may cause starvation

3) Round robin is better than FCFS in terms of response time

(A) 1 only **(B)** 1 and 3 only **(C)** 2 and 3 only **(D)** 1, 2 and 3

Q An example of a non-preemptive CPU scheduling algorithm is: **(NET-June-2008)**

(A) Shortest job first scheduling.

(B) Round robin scheduling.

(C) Priority scheduling.

(D) Fair share scheduling.

Q Consider three processes, all arriving at time zero, with total execution time of 10, 20 and 30 units, respectively. Each process spends the first 20% of execution time doing I/O, the next 70% of time doing computation, and the last 10% of time doing I/O again. The operating system uses a shortest remaining compute time first scheduling algorithm and schedules a new process either when the running process gets blocked on I/O or when the running process finishes its compute burst. Assume that all I/O operations can be overlapped as much as possible. For what percentage of time does the CPU remain idle? **(GATE-2006) (2 Marks)**

(A) 0% **(B)** 10.6% **(C)** 30.0% **(D)** 89.4%

Break

Q The arrival time, priority, and duration of the CPU and I/O bursts for each of three processes P_1, P_2 and P_3 are given in the table below. Each process has a CPU burst followed by an I/O burst followed by another CPU burst. Assume that each process has its own I/O resource. **(GATE-2006) (2 Marks)**

Process	Arrival Time	Priority	Burst duration, CPU, I/O CPU
P_1	0	2	1,5,3
P_2	2	3(L)	3,3,1
P_3	3	1(H)	2,3,1

The programmed operating system uses preemptive priority scheduling. What are the finish times of the processes P_1, P_2 and P_3 ?

(A) 11, 15, 9 **(B)** 10, 15, 9 **(C)** 11, 16, 10 **(D)** 12, 17, 11

Q .....is one of pre-emptive scheduling algorithm. **(NET-Dec-2006)**

(A) Shortest-Job-first

(B) Round-robin

(C) Priority based

(D) Shortest-Job-next

Q A uniprocessor computer system only has two processes, both of which alternate 10 ms CPU bursts with 90 ms I/O bursts. Both the processes were created at nearly the same time. The I/O of both processes can proceed in parallel. Which of the following scheduling strategies will result in the least CPU utilization (over a long period of time) for this system? **(GATE-2003) (2 Marks)**

(A) First come first served scheduling

(B) Shortest remaining time first scheduling

(C) Static priority scheduling with different priorities for the two processes

(D) Round robin scheduling with a time quantum of 5 ms

Break

Longest Job First

- Process having longest Burst Time will get scheduled first.
- It can be both pre-emptive and non-pre-emptive in nature.
- The pre-emptive version is referred to as Longest Remaining Time First (LRTF) Scheduling Algorithm.

P. No	AT	BT
P_1	0	3
P_2	1	2
P_3	2	4
P_4	3	5
P_5	4	6

Break

Longest Remaining Time First (LRTF)

- It is pre-emptive in nature.
- Longest Burst time process will get scheduled first.

P. No	AT	BT
P_1	0	2
P_2	0	4
P_3	0	8

Q Consider three processes (process id 0, 1, 2 respectively) with compute time bursts 2, 4 and 8 time units. All processes arrive at time zero. Consider the longest remaining time first (LRTF) scheduling algorithm. In LRTF ties are broken by giving priority to the process with the lowest process id. The average turnaround time is? **(GATE-2006) (2 Marks)**

P. No	AT	BT	CT	TAT
P_0	0	2		
P_1	0	4		
P_2	0	8		

(A) 13 units **(B)** 14 units **(C)** 15 units **(D)** 16 units

Break

Highest response ratio next (HRRN)

- Scheduling is a non-pre-emptive discipline, similar to shortest job next (SJN), in which the priority of each job is dependent on its estimated run time, and also the amount of time it has spent waiting.

- Jobs gain higher priority the longer they wait, which prevents indefinite waiting or in other words what we say starvation. In fact, the jobs that have spent a long time waiting compete against those estimated to have short run times.
- Response Ratio = (W + S)/S

- Here, **W** is the waiting time of the process so far and **S** is the Burst time of the process.

- So, the conclusion is it gives priority to those processes which have less burst time (or execution time) but also takes care of the waiting time of longer processes, thus preventing starvation.

Q Consider a set of n tasks with known runtimes r_1, r_2, …. r_n to be run on a uniprocessor machine. Which of the following processor scheduling algorithms will result in the maximum throughput? **(GATE-2001) (1 Marks)**

(A) Round-Robin

(B) Shortest-Job-First

(C) Highest-Response-Ratio-Next

(D) First-Come-First-Served

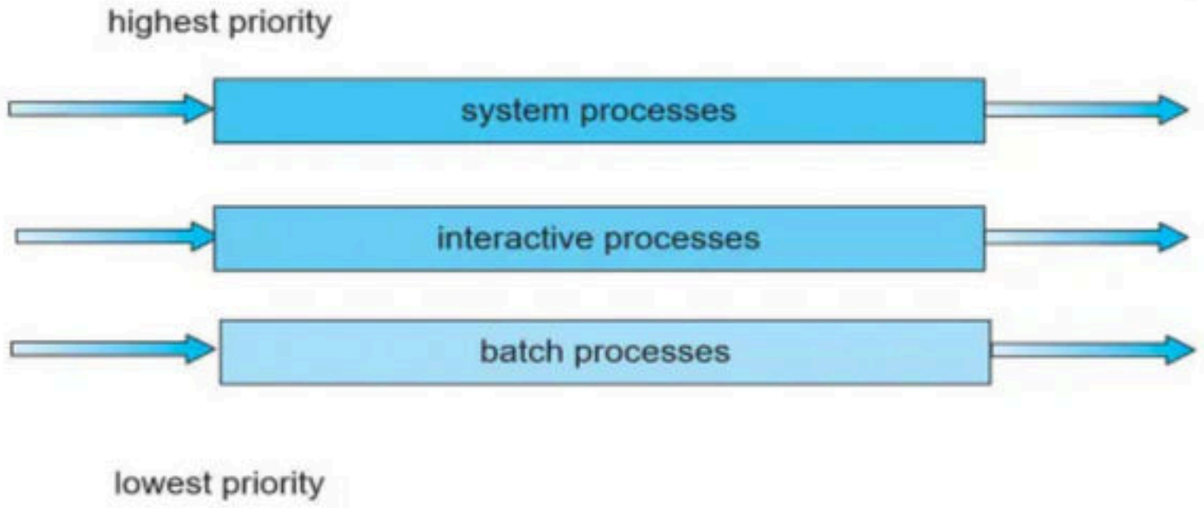
Q highest response ratio next scheduling policy favours ------- jobs, but it also limits the waiting time of ---------- jobs. **(GATE-1990) (1 Marks)**

Break

Multi Level-Queue Scheduling

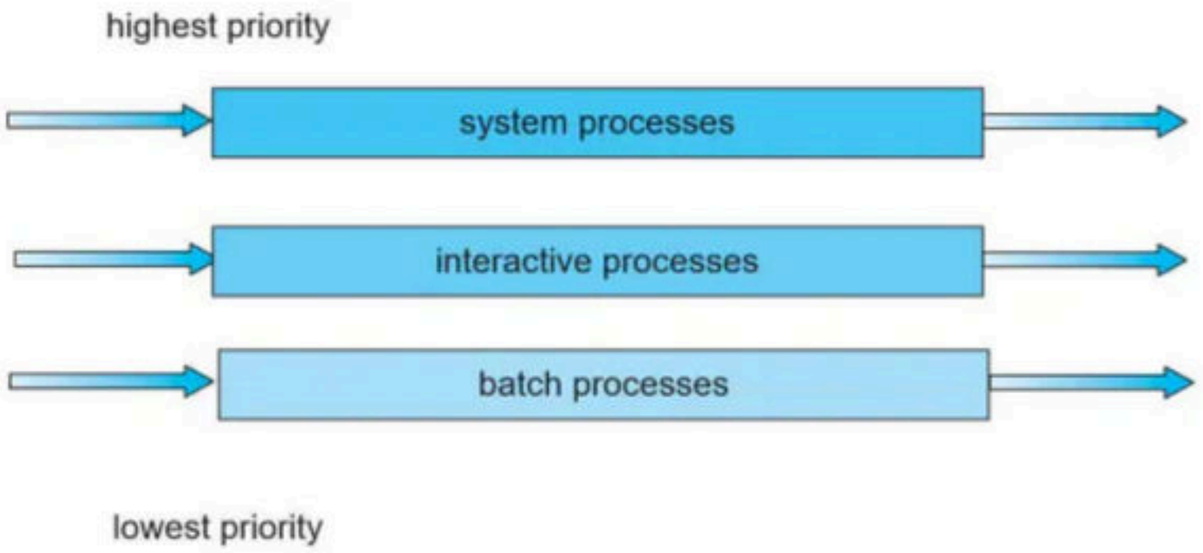
- Another class of scheduling algorithm has been created for situations in which processes are easily classified into different groups. For example, a common division is made between **foreground** (interactive) processes and **background** (batch) processes. These two types of processes have different response-time requirements and so may have different scheduling needs.

- A **multilevel queue** scheduling algorithm partitions the ready queue into several separate queues. The processes are permanently assigned to one queue, generally based on some property of the process, such as memory size, process priority, or process type.



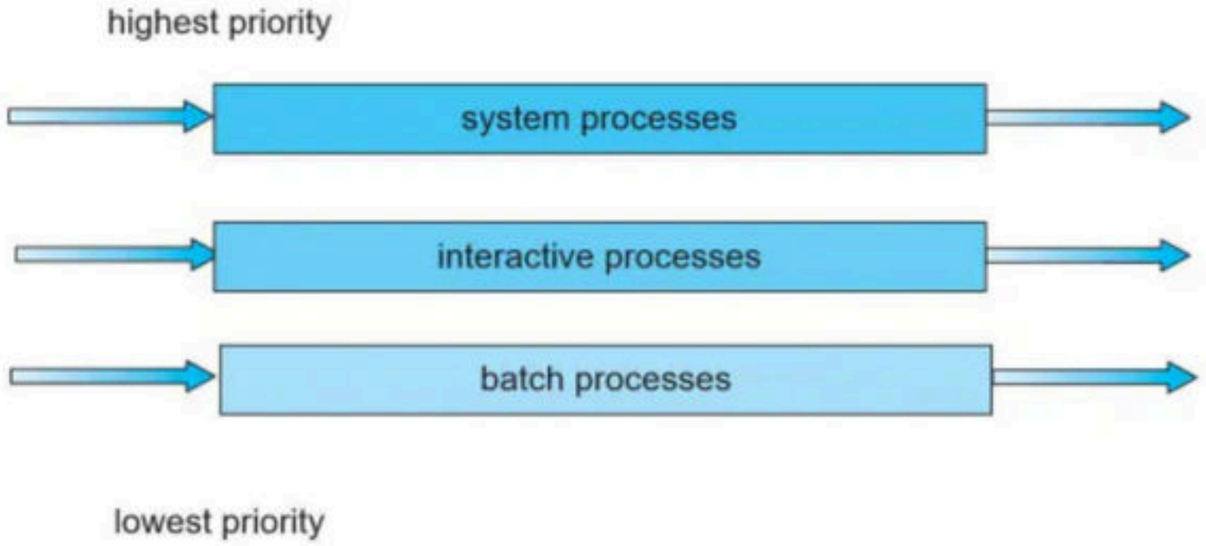
- Each queue has its own scheduling algorithm. For example, separate queues might be used for foreground and background processes. The foreground queue might be scheduled by an RR algorithm, while the background queue is scheduled by an FCFS algorithm.

- In addition, there must be scheduling among the queues, which is commonly implemented as fixed-priority preemptive scheduling. For example, the foreground queue may have absolute priority over the background queue.



- Each queue has absolute priority over lower-priority queues. No process in the batch queue, for example, could run unless the queues for system processes, interactive processes, and interactive editing processes were all empty. If an interactive editing process entered the ready queue while a batch process was running, the batch process would be preempted.

- Another possibility is to time-slice among the queues. Here, each queue gets a certain portion of the CPU time, which it can then schedule among its various processes. For instance, in the foreground– background queue example, the foreground queue can be given 80 percent of the CPU time for RR scheduling among its processes, while the background queue receives 20 percent of the CPU to give to its processes on an FCFS basis.



Q Consider the following justifications for commonly using the two-level CPU scheduling:

I. It is used when memory is too small to hold all the ready processes.

II. Because its performance is same as that of the FIFO.

III. Because it facilitates putting some set of processes into memory and a choice is made from that.

IV. Because it does not allow to adjust the set of in-core processes.

Which of the following is true? **(NET-Dec-2014)**

(A) I, III and IV **(B)** I and II **(C)** III and IV **(D)** I and III

Break

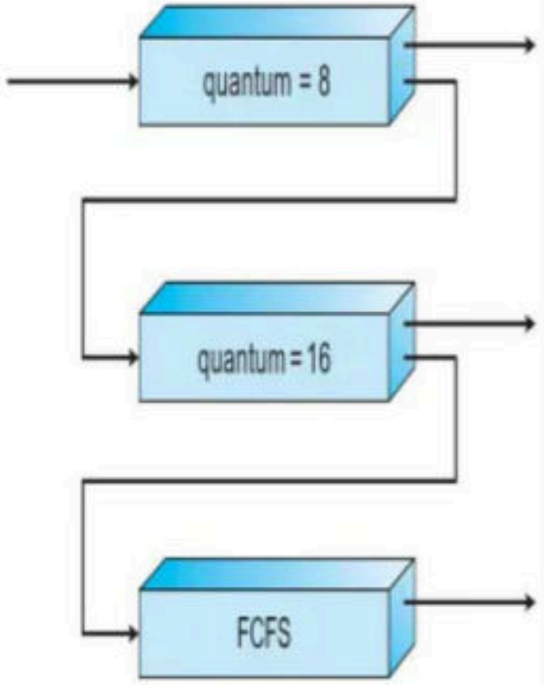
Multi-level Feedback Queue Scheduling

- Normally, when the multilevel queue scheduling algorithm is used, processes are permanently assigned to a queue when they enter the system. If there are separate queues for foreground and background processes, for example, processes do not move from one queue to the other, since processes do not change their foreground or background nature. This setup has the advantage of low scheduling overhead, but it is inflexible.

- The **multilevel feedback queue** scheduling algorithm, in contrast, allows a process to move between queues. The idea is to separate processes according to the characteristics of their CPU bursts. If a process uses too much CPU time, it will be moved to a lower-priority queue. This scheme leaves I/O-bound and interactive processes in the higher-priority queues.

- In addition, a process that waits too long in a lower-priority queue may be moved to a higher-priority queue. This form of aging prevents starvation.

- A process entering the ready queue is put in queue 0.A process in queue 0 is given a time quantum of 8 milliseconds. If it does not finish within this time, it is moved to the tail of queue 1. If queue 0 is empty, the process at the head of queue 1 is given a quantum of 16 milliseconds. If it does not complete, it is preempted and is put into queue 2.



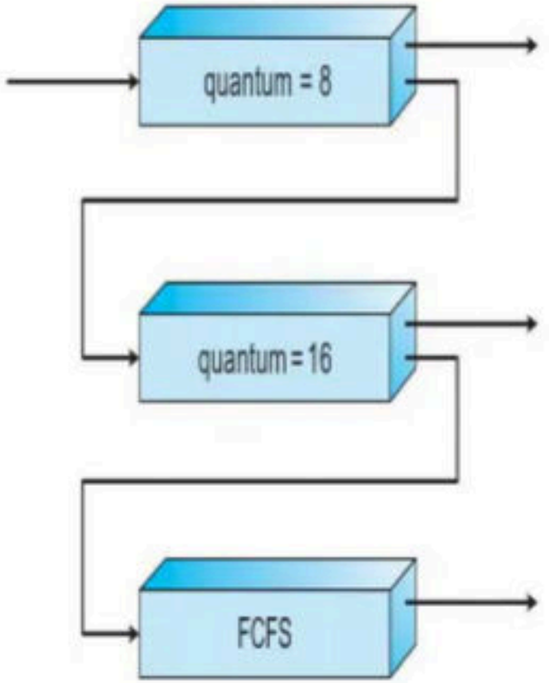
- Processes in queue 2 are run on an FCFS basis but are run only when queues 0 and 1 are empty.

- This scheduling algorithm gives highest priority to any process with a CPU burst of 8 milliseconds or less. Such a process will quickly get the CPU, finish its CPU burst, and go off to its next I/O burst.

- Processes that need more than 8 but less than 24 milliseconds are also served quickly, although with lower priority than shorter processes. Long processes automatically sink to queue 2 and are served in FCFS order with any CPU cycles left over from queues 0 and 1.

In general, a multilevel feedback queue scheduler is defined by the following parameters:

- The number of queues
- The scheduling algorithm for each queue
- The method used to determine when to upgrade a process to a higher- priority queue
- The method used to determine when to demote a process to a lower- priority queue
- The method used to determine which queue a process will enter when that process needs service
- The definition of a multilevel feedback queue scheduler makes it the most general CPU-scheduling algorithm. It can be configured to match a specific system under design. Unfortunately, it is also the most complex algorithm,
- since defining the best scheduler requires some means by which to select values for all the parameters.



Break

Q Which of the following statements is not true for Multi-Level Feedback Queue processor scheduling algorithm? **(NET-June-2015)**

a) Queues have different priorities

b) Each queue may have different scheduling algorithm

c) Processes are permanently assigned to a queue

d) This algorithm can be configured to match a specific system under design

Q Match the following:

List – I	List – II
a. Multilevel feedback queue	i. Time-slicing
b. FCFS	ii. Criteria to move processes between queues
c. Shortest process next	iii. Batch processing
d. Round robin scheduling	iv. Exponential smoothening

Codes: **(NET-June-2014)**

	a	b	c	d
(A)	i	iii	ii	iv
(B)	iv	iii	ii	i
(C)	iii	i	iv	i
(D)	ii	iii	iv	i

Q An example of a non-pre-emptive scheduling algorithm is: **(NET-Dec-2008)**

(A) Round Robin

(B) Priority Scheduling

(C) Shortest job first

(D) 2 level scheduling

Q Which of the following scheduling algorithms is non-preemptive? **(GATE-2002) (1 Marks)**

(A) Round Robin

(B) First-In First-Out

(C) Multilevel Queue Scheduling

(D) Multilevel Queue Scheduling with Feedback

Break

Q An Operating System (OS) crashes on the average once in 30 days, that is, the Mean Time Between Failures (MTBF) = 30 days. When this happens, it takes 10 minutes to recover the OS, that is, the Mean Time To Repair (MTTR) = 10 minutes. The availability of the OS with these reliability figures is approximately: **(NET-AUG-2016)**

a) 96.97% **b)** 97.97% **c)** 99.009% **d)** 99.97%

Q Consider a uniprocessor system where new processes arrive at an average of five processes per minute and each process needs an average of 6 seconds of service time. What will be the CPU utilization? **(NET-JUNE-2014)**

a) 80 % **b)** 50 % **c)** 60 % **d)** 30 %

Q.4 Three processes arrive at time zero with CPU bursts of 16, 20 and 10 milliseconds. If the scheduler has prior knowledge about the length of the CPU bursts, the minimum achievable average waiting time for these three processes in a non-preemptive scheduler (round to nearest integer) is _____ milliseconds.

(Gate-2021) (1 Marks)

Q.19 Which of the following statement(s) is/are correct in the context of CPU scheduling? **(GATE-2021)**

(a) The goal is to only maximize CPU utilization and minimize throughput.

(b) Turnaround time includes waiting time.

(c) Round-robin policy can be used even when the CPU time required by each of the processes is not known apriori.

(d) Implementing preemptive scheduling needs hardware support.